

Dokumentacja: HTML i CSS w React

Dokumentacja ta stanowi kompleksowe źródło wiedzy na temat tworzenia nowoczesnych interfejsów użytkownika przy użyciu HTML i CSS, ze szczególnym uwzględnieniem specyfiki pracy w bibliotece React (JSX). Skierowana jest zarówno do osób rozpoczynających naukę, jak i zaawansowanych programistów poszukujących referencyjnych informacji o tagach, atrybutach i zaawansowanych wzorcach.

Spis treści

1. Wprowadzenie do HTML
2. Tagi tekstowe i nagłówki
3. Listy w HTML
4. Linki i nawigacja
5. Obrazy i multimedia
6. Tabele w HTML
7. Formularze w HTML
8. Elementy semantyczne HTML5
9. Atrybuty globalne i specjalne
10. HTML w React — JSX
11. Wprowadzenie do CSS
12. Selektory CSS
13. Jednostki i wartości CSS
14. Typografia i tekst w CSS
15. Model pudełkowy (Box Model)
16. Flexbox i Grid
17. Pozycjonowanie elementów
18. Responsywność (RWD) i Media Queries
19. Przejścia CSS (Transitions) w Praktyce
20. Animacje i Keyframes
21. Transformacje 2D i 3D
22. Tła, Gradienty i Filtry Wizualne
23. Zmienne CSS i Dynamiczny Motyw (Dark Mode)
24. Wzorce Układów: Flexbox Masterclass
25. Wzorce Układów: CSS Grid Recipes
26. Style w React: Moduły i CSS-in-JS
27. Walidacja Formularzy i API Przeglądarki
28. SVG i Element Canvas
29. Wydajność (Performance) i Dostępność (A11y)
30. Budowa Własnego UI Kit w React (Tailwind CSS)
31. Zaawansowane Komponenty (Dropdown, Accordion, Zakładki)
32. Zaawansowana Kontrola Rozmieszczenia (Stacking Context)
33. Scroll-Driven Animations (Animacje Napędzane Scrollowaniem)

1. Wprowadzenie do HTML

1.1. Czym jest HTML

HTML (HyperText Markup Language) to standardowy język znaczników używany do tworzenia stron internetowych. Nie jest to język programowania (nie posiada pętli, instrukcji warunkowych ani zmiennych w klasycznym sensie), lecz język opisujący strukturę i semantykę zawartości.

Przeglądarka internetowa (np. Chrome, Firefox, Safari) pobiera dokument HTML z serwera, a następnie interpretuje go (parsuje), budując w pamięci strukturę drzewiastą zwaną **DOM** (Document Object Model). Na podstawie DOM przeglądarka renderuje stronę na ekranie.

1.2. Struktura dokumentu HTML

Każdy prawidłowy dokument HTML5 musi posiadać określoną strukturę bazową.

```
<!DOCTYPE html>

<html lang="pl">

<head>

  <!-- Sekcja nagłówkowa (metadane) - niewidoczna dla użytkownika -->

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Tytuł Strony (widoczny w zakładce przeglądarki)</title>

  <!-- Linkowanie zewnętrznego arkusza stylów -->

  <link rel="stylesheet" href="style.css">

  <!-- Dołączenie skryptu JS -->

  <script src="script.js" defer></script>

</head>

<body>

  <!-- Sekcja body - to co widzi użytkownik -->

  <h1>Witaj na mojej stronie</h1>

  <p>To jest pierwszy paragraf.</p>

</body>

</html>
```

NAJWAŻNIEJSZE ELEMENTY SEKCJI <HEAD>

Elementy wewnątrz <head> dostarczają przeglądarce i wyszukiwarkom instrukcji dotyczących sposobu przetwarzania strony.

TAGI	ATRYBUTY	OPIS	PRZYKŁAD
<title>	-	Określa tytuł strony (pasek zakładek, wyniki wyszukiwania).	<title>Sklep</title>
<meta>	charset="UTF-8"	Deklaruje kodowanie znaków (kluczowe dla polskich znaków).	<meta charset="UTF-8">
<meta>	name="viewport"	Kontroluje skalowanie strony na urządzeniach mobilnych (RWD).	<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta>	name="description"	Krótki opis strony widoczny pod linkiem w wyszukiwarkach (SEO).	<meta name="description" content="Nasz sklep z butami">
<link>	rel="stylesheet" href="..."	Linkuje zewnętrzne zasoby (głównie pliki CSS i ikony).	<link rel="stylesheet" href="main.css">
<link>	rel="icon" href="..."	Określa ikonę strony (Favicon).	<link rel="icon" href="favicon.ico">
<script>	src="..." defer async	Ładuje kod JavaScript. Atrybut defer opóźnia wykonanie po HTML.	<script src="app.js" defer></script>
<style>	-	Pozwala pisać kod CSS bezpośrednio w pliku HTML.	<style> body { color: red; } </style>

1.3. Znaczniki i atrybuty

Elementy HTML są zapisywane przy pomocy **znaczników** (tagów). Większość z nich składa się z tagu otwierającego i zamykającego: <nazwatagu>Treść elementu</nazwatagu>

Niektóre tagi są **samozamykające się** (void elements), co oznacza, że nie mają zawartości i zamykają się same w sobie: ``, `<br>`, `<hr>`, `<input>`, `<meta>`. W standardzie HTML5 nie musimy używać ukośnika na końcu (choć `<br />` jest dozwolone; w React/JSX jest wymagane).

ATRYBUTY GLOBALNE

Atrybuty modyfikują zachowanie tagów lub dostarczają im dodatkowych danych. Umieszczamy je ZAWSZE w tagu otwierającym.

ATRYBUT	OPIS	PRZYKŁAD UŻYCIA
id	Unikalny identyfikator elementu na stronie. Nie może się powtarzać. Używany w JS i CSS.	<code><div id="header"></code>
class	Klasa elementu. Może być przypisana do wielu elementów. Używana gł. do CSS.	<code><p class="tekst error"></code>
style	Style inline. CSS nakładany bezpośrednio na element.	<code></code>
title	Tekst podpowiedzi (tooltip) pojawiający się po najechaniu myszką.	<code></code>
lang	Kod języka dla elementu (ważne dla czytników ekranowych i SEO).	<code><html lang="pl"></code>
hidden	Ukrywa element całkowicie przed przeglądarką (nie zajmuje miejsca).	<code><div hidden></code>
tabindex	Kolejność elementu podczas nawigacji klawiszem Tab.	<code><button tabindex="1"></code>
data-*	Własne atrybuty pozwalające przechowywać dodatkowe dane dostępne w JS.	<code><li data-id="5" data-category="books"></code>

1.4. Komentarze i Encje

Komentarze w HTML służą do pozostawiania notatek dla innych programistów lub tymczasowego wyłączenia fragmentów kodu.

```

<!-- To jest komentarz w HTML. Przeglądarka go nie wyrenderuje. -->
<p>To jest widoczne.</p>
<!-- <p>To zostało tymczasowo ukryte.</p> -->
```

Encje HTML (Entities) Niektóre znaki mają w HTML specjalne znaczenie (np. `<` i `>`). Jeśli chcemy wyświetlić je w tekście jako zwykły znak, musimy użyć odpowiednich encji. Przydają się też do wstawiania znaków specjalnych.

ZNAK	ZNACZENIE W TEKŚCIE	ENCJA (KOD)	ZASTOSOWANIE
<	Znak mniejszości (początek tagu)	<	Pokazywanie kodu: <code>&lt;div&gt;</code>
>	Znak większości (koniec tagu)	>	Pokazywanie kodu
&	Ambersand	&	Firmy np. "A & B"
"	Cudzysłów	"	Wewnątrz atrybutów
'	Apostrof	'	-
(spacja)	Niełamiąca się spacja	 	Zapobiega łamaniu wiersza (np. "15 kg")
©	Znak praw autorskich	©	W stopce np. <code>&copy; 2024</code>
®	Zastrzeżony znak towarowy	®	Przy nazwach marek
™	Znak towarowy	™	-
€	Euro	€	Ceny
×	Mnożenie / Krzyżyk zamykania	×	Klawisze zamknięcia modala (✖)

2. Tagi tekstowe i nagłówki

2.1. Nagłówki (`<h1>` – `<h6>`)

HTML oferuje 6 poziomów nagłówków. Służą one do budowania logicznej hierarchii dokumentu, co jest krytyczne dla osób korzystających z czytników ekranowych oraz dla pozycjonowania (SEO).

```
<h1>Główny tytuł strony (powinien być tylko jeden)</h1>
<h2>Podtytuł sekcji</h2>
<h3>Nagłówek mniejszej części wewnątrz h2</h3>
<h4>Jeszcze mniejszy nagłówek</h4>
<h5>Bardzo mały nagłówek</h5>
<h6>Najmniejszy nagłówek (rzadko używany)</h6>
```

Ważne zasady:

1. Nie używaj nagłówków tylko do tego, żeby tekst był duży/pogrubiony. Od tego jest CSS (`font-size` , `font-weight`).
2. Utrzymuj hierarchię. Nie skacz z `<h2>` od razu na `<h4>` z pominięciem `<h3>` .
3. Najlepiej, aby na stronie był dokładnie jeden `<h1>` precyzyjnie opisujący jej treść.

2.2. Paragrafy i łamanie wierszy

Tag `<p>` (paragraph) oznacza akapit tekstu. Przeglądarka domyślnie dodaje margines górny i dolny przed i po każdym akapicie.

```
<p>To jest pierwszy akapit. Może zawierać wiele zdań, a tekst będzie się automatycznie łamał po dojściu do krawędzi ekranu.</p>
<p>To jest drugi akapit. Jest oddzielony od pierwszego widocznym odstępem.</p>
```

Znaczniki przerwania:

- `<br>` (Line Break): Łamie linię twardo. Zwykły znak Enter w edytorze kodu jest przez HTML traktowany jako spacja. Używamy `<br>` np. przy pisaniu wierszy czy adresów pocztowych.
- `<hr>` (Horizontal Rule): Znacznik blokowy tworzący poziomą linię, reprezentującą tematyczne oddzielenie treści (np. nowa scena w opowiadaniu).

```
<p>
  Jan Kowalski<br>
  ul. Kwiatowa 15<br>
  00-001 Warszawa
</p>
<hr>
<p>Kolejna sekcja strony</p>
```

2.3. Tagi formatujące tekst (inline)

Elementy formatujące zazwyczaj nie tworzą nowych linii (są typu *inline*). HTML5 kładzie duży nacisk na ich znaczenie **semantyczne** (jak czytnik i wyszukiwarka to rozumieją), a nie tylko na wygląd.

TAG	WYGLĄD DOMYŚLNY	ZNACZENIE SEMANTYCZNE
<code></code>	Pogrubiony	Bardzo ważny tekst (np. ostrzeżenie).
<code></code>	Pogrubiony	Zwykłe wizualne pogrubienie (bez specjalnej wagi).
<code></code>	<i>Pochylony</i>	Tekst zaakcentowany (emfaza) (czytnik czyta to innym tonem).
<code><i></code>	<i>Pochylony</i>	Zwykłe pochYLENIE (często używane do nazw, terminów obcych, lub... ikon).
<code><mark></code>	Podświetlony	Tekst zaznaczony/wyróżniony, jak zakreślaczem.
<code></code>	Przekreślony	Tekst usunięty / nieaktualny. Często z atrybutem <code>datetime</code> .
<code><ins></code>	<u>Podkreślony</u>	Tekst świeżo wstawiony. Oznaczenie poprawek do <code></code> .
<code><sub></code>	Indeks dolny	np. we wzorach chemicznych: H ₂ O
<code><sup></code>	Indeks górny	np. matematyka: E = mc ²
<code><small></code>	Mniejszy druk	Druk drobny (tzw. fine print), np. w przypisach prawnych, copyright.
<code><code></code>	Font o stałej szer.	Oznacza fragment kodu źródłowego programu w tekście.
<code><kbd></code>	Font o stałej szer.	Wciskany klawisz klawiatury np. <code>Ctrl</code> + <code>C</code> .
<code><abbr></code>	Zwyczaj z kropkami	Skrót, rozwinięcie z atrybutem <code>title</code> . np. <code><abbr title="World Health Organization">WHO</abbr></code> .

2.4. Tekst preformatowany (`<pre>`)

Przeglądarki HTML "zjadają" wszystkie wielokrotne spacje i entery, redukując je do jednej spacji (tzw. white-space collapsing). Tag `<pre>` zapobiega temu. Często łączy się go z `<code>` do wyświetlania bloków kodu.

```
<pre>
<code>
function hello() {
    console.log("Ten kod zachowa wcięcia!");
}
</code>
</pre>
```

2.5. Blokowe cytaty (`<blockquote>` i `<q>`)

Służą do poprawnego oznaczania cytatów z innych źródeł. Można użyć atrybutu `cite` do podania adresu URL źródła.

```
←!— Długi cytat w osobnym bloku —→
<blockquote cite="https://pl.wikipedia.org/wiki/HTML">
    <p>HTML - hipertekstowy język znaczników, wykorzystywany do tworzenia dokumentów hipertekstowych.</p>
</blockquote>

←!— Krótki cytat w tekście —→
<p>Jak powiedział Yoda: <q>Rób. Albo nie rób. Nie ma próbowania.</q></p>
```

Tag `<q>` domyślnie sam dodaje wokół tekstu znaki cudzysłowu.

2.6. Najważniejsze tagi ogólnego przeznaczenia: `<div>` i `<span>`

Są to tzw. puste "kontenery", które nie niosą żadnego znaczenia semantycznego. Są niezbędne do ostylowania elementów w CSS oraz programowania logiki z JavaScript.

- `<div>` (Divider): Element blokowy (`display: block`). Zajmuje całą szerokość dostępnej linii, spychając kolejne elementy pod spód. Tworzy duże sekcje / pudła na stronie.
- `<span>`: Element liniowy (`display: inline`). Wpasowuje się w tekst. Używamy go do np. zmiany koloru pojedynczego słowa.

```
<div class="card" style="border: 1px solid black; padding: 10px;">
  <h2>Karta produktu</h2>
  <p>Cena promocyjna to <span style="color: red; font-weight: bold;">99,00 zł</span>.</p>
</div>
```

3. Listy w HTML

Listy to podstawa uporządkowywania informacji. Używa się ich nie tylko do zwykłych "wypunktowań", ale także do tworzenia menu nawigacyjnych na stronach.

3.1. Lista nieuporządkowana (wypunktowana) - `<ul>`

Znacznik `<ul>` (Unordered List) otacza całą listę. Poszczególne elementy wewnątrz to `<li>` (List Item). Domyślnie każdy punkt to czarna kropka (bullet).

```
<h3>Składniki na pizzę:</h3>
<ul>
  <li>Mąka typ 00</li>
  <li>Woda (letnia)</li>
  <li>Drożdże</li>
  <li>Sól</li>
</ul>
```

Tip: Możemy modyfikować wygląd punktorów w CSS używając właściwości `list-style-type`. Oto najpopularniejsze wartości dla list nieuporządkowanych (`<ul>`):

- `list-style-type: disc;` (domyślna pełna czarna kropka)
- `list-style-type: circle;` (kropka pusta w środku)
- `list-style-type: square;` (pełny czarny kwadrat)
- `list-style-type: none;` (całkowite usunięcie punktorów – obowiązkowe przy tworzeniu z list np. pasków nawigacyjnych).

3.2. Lista uporządkowana (numerowana) - `<ol>`

Znacznik `<ol>` (Ordered List) automatycznie numeruje elementy `<li>`.

```
<h3>Jak zrobić herbatę:</h3>
<ol>
  <li>Zalej wodę do czajnika.</li>
  <li>Zagotuj wodę.</li>
  <li>Włóż torebkę herbaty do kubka.</li>
  <li>Zalej gorącą wodą.</li>
</ol>
```

Tip: Właściwość `list-style-type` pozwala również zmieniać sposób numerowania w listach uporządkowanych (`<ol>`). Najważniejsze wartości to:

- `list-style-type: decimal;` (domyślne liczby arabskie: 1, 2, 3...)

- `list-style-type: lower-alpha;` (małe litery: a, b, c...)
- `list-style-type: upper-alpha;` (wielkie litery: A, B, C...)
- `list-style-type: lower-roman;` (małe liczby rzymskie: i, ii, iii...)
- `list-style-type: upper-roman;` (wielkie liczby rzymskie: I, II, III...)

Atrybuty specyficzne dla `<ol>`:

ATRYBUT	WARTOŚĆ/OPIS	PRZYKŁAD UŻYCIA
<code>type</code>	Jakiego formatu używać do numeracji. <code>1</code> (arabskie), <code>A</code> lub <code>a</code> (litery), <code>I</code> lub <code>i</code> (rzymskie).	<code><ol type="I"></code>
<code>start</code>	Od jakiego numeru zacząć odliczanie.	<code><ol start="5"></code>
<code>reversed</code>	Odliczanie w dół. Atrybut logiczny (bez wartości).	<code><ol reversed></code>

```

<!-- Lista numerowana A, B, C zaczynająca się od litery 'C' (3 litera) -->
<ol type="A" start="3">
  <li>Opcja C</li>
  <li>Opcja D</li>
</ol>
```

3.3. Lista definicji - `<dl>`

Służy do tworzenia glosariuszy, słowników terminów i pojęć.

- `<dl>` (Description List) - Kontener
- `<dt>` (Description Term) - Pojęcie / termin
- `<dd>` (Description Details) - Definicja (domyślnie wcięta)

```

<dl>
  <dt>HTML</dt>
  <dd>Język znaczników do tworzenia stron WWW.</dd>

  <dt>CSS</dt>
  <dd>Arkusze stylów do opisu wyglądu strony.</dd>
</dl>
```

3.4. Listy zagnieżdżone

Elementem dzieckiem `<ul>` lub `<ol>` może być TYLKO `<li>`. Jeśli chcemy stworzyć pod-listę, umieszczamy drugą listę `<ul>` wewnątrz elementu `<li>`.

```

<ul>
  <li>Programowanie Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Programowanie Backend
    <ul>
      <li>Node.js</li>
    </ul>
  </li>
</ul>
```

```
</li>Python</li>
</ul>
</li>
</ul>
```

W powyższym przykładzie przeglądarka automatycznie użyje wcięć oraz zmieni kropki w pod-listach na puste kółka.

4. Linki i nawigacja

Tag `<a>` (Anchor) jest podstawą funkcjonowania sieci WWW - pozwala użytkownikom przenosić się między zasobami.

4.1. Tworzenie linków i atrybut href

Najważniejszym atrybutem jest `href` (Hypertext Reference). Określa on, dokąd prowadzi dany link.

```
←!— Link absolutny (prowadzi do innej strony w sieci) →
<a href="https://google.com">Przejdź do wyszukiwarki Google</a>

←!— Link względny/relatywny (prowadzi do innego pliku na tym samym serwerze) →
<a href="/o-nas.html">Przeczytaj o naszej firmie</a>
<a href="../kontakt.html">Przejdź katalog wyżej do kontaktu</a>
```

4.2. Atrybut target

Kontroluje, gdzie ma zostać otwarta podlinkowana strona.

WARTOŚĆ	ZNACZENIE
<code>_self</code>	(Domyślne) Otwiera stronę w obecnej karcie.
<code>_blank</code>	Otwiera stronę w nowej karcie/oknie przeglądarki.
<code>_parent</code> / <code>_top</code>	Używane rzadko, przy wewnątrz ramkach (<code><iframe></code>).

Zasada bezpieczeństwa: Jeśli otwierasz zewnętrzną stronę z `target="_blank"`, zawsze dodawaj atrybut `rel="noopener noreferrer"`. Uniemożliwi to złośliwej stronie dostęp do obiektu `window` Twojej strony (co mogłoby być użyte do przejęcia jej - atak tabnabbing).

```
<a href="https://nieznanastrona.com" target="_blank" rel="noopener noreferrer">Kliknij tutaj</a>
```

4.3. Linki do plików, e-maila i telefonu

Oprócz adresów stron (`http://`), `href` obsługuje też inne protokoły:

```
←!— Pobieranie pliku (atrybut download nakazuje pobranie bez jego podglądu) →
<a href="regulamin.pdf" download="RegulaminSklepu.pdf">Pobierz regulamin</a>

←!— Otwieranie domyślnego klienta poczty z gotowym adresem →
<a href="mailto:kontakt@firma.pl">Napisz e-mail</a>

←!— Otwieranie klienta poczty z predefiniowanym tematem →
<a href="mailto:admin@domena.pl?subject=Zgłoszenie błędu na stronie">Zgłoś błąd</a>
```


←!— Wywoływanie aplikacji do dzwonienia na smartfonach →

```
<a href="tel:+48111222333">Zadzwoń: 111-222-333</a>
```

←!— Otwarcie SMS →

```
<a href="sms:+48111222333">Wyślij SMS</a>
```

4.4. Kotwice na stronie (Skoki do elementów)

Możesz przewijać użytkownika gładko w inne miejsce **na tej samej stronie**. Wymaga to dwóch kroków: nadania celowi unikalnego atrybutu `id`, a następnie użycia `#nazwa-id` w linku.

←!— Miejsce skoku (gdzieś na dole strony) →

```
<h2 id="kontakt-sekcja">Sekcja Kontaktowa</h2>
```

```
<p>Zadzwoń do nas!</p>
```

←!— Link przenoszący do tego miejsca (gdzieś u góry strony) →

```
<a href="#kontakt-sekcja">Przejdź do kontaktu</a>
```

←!— Przewinięcie na samą górę strony →

```
<a href="#">Wróć na górę</a>
```

Tip: Aby przewijanie było płynne, dodaj do CSS: `html { scroll-behavior: smooth; }`

5. Obrazy i multimedia

5.1. Wstawianie obrazów (`<img>`)

Tag `<img>` jest elementem samozamykającym. Jest to tag **inline-block** (ustawia się obok tekstu, ale można mu narzucić szerokość/wysokość).

```

```

Atrybuty obrazu:

ATRYBUT	OPIS I ZACHOWANIE
<code>src</code>	Źródło obrazka. URL bezwzględny lub ścieżka względna na serwerze. (Wymagany)
<code>alt</code>	Tekst alternatywny. Niezwykle ważny dla SEO i czytników ekranu (niewidomych). Pokazuje się również, gdy obrazka nie uda się załadować. (Wymagany)
<code>width</code> , <code>height</code>	Opcjonalna stała szerokość i wysokość (w pikselach). Mimo że częściej styluje się w CSS, podanie ich tutaj w HTML zapobiega skakaniu strony (Layout Shift), rezerwując miejsce podczas ładowania.
<code>loading</code>	Kontroluje strategię ładowania. <code>lazy</code> – pobierz dopiero gdy obraz ma pojawić się na ekranie (super dla wydajności dłuugich stron). <code>eager</code> – domyślnie, ładuj od razu.
<code>title</code>	Tekstowy tooltip pojawiający się po najechaniu kursorem na obraz.

5.2. Formaty obrazów na stronach WWW

Różne formaty służą do różnych celów. Obecnie nowoczesne strony mocno faworyzują format WebP zamiast PNG/JPG ze względu na drastycznie mniejszy rozmiar.

FORMAT	PRZEZROCZYSTOŚĆ	KOMPRESJA (STRATY)	UŻYCIĘ
JPEG / JPG	✗	Stratna (niszczy detale)	Zdjęcia, bogate w szczegóły ilustracje (najiższy z old-schoolowych)
PNG	✓	Bezstratna	Zrzuty ekranu, loga, tekst na obrazie, tła bez tła
GIF	✓ (ale słaba)	Bezstratna (256 kolorów!)	Animowane memy. Bardzo przestarzały technologicznie format.
SVG	✓	Wektorowa (kod matematyczny)	Logotypy, ikony. Skaluje się w nieskończoność bez utraty jakości. Kod SVG to tak naprawdę XML (tekst).
WebP	✓	Stratna lub Bezstratna	Nowoczesny format (od Google). Zastępuje i JPG i PNG ze znacznie mniejszym rozmiarem.
AVIF	✓	-	Super nowość, jeszcze lepsza kompresja niż WebP.

5.3. Element `figure` i podpisy

Znaczniki te powiązują ze sobą obrazek z jego opisem (np. "Ryc. 1" w książce). Semantycznie są ze sobą sklejone.

```
<figure>
  
  <figcaption>Fot. 1.1: Golden Retriever na letniej łące. Źródło: własne.</figcaption>
</figure>
```

5.4. Osadzanie wideo (`<video>`) i audio (`<audio>`)

HTML5 wprowadził natywną obsługę wideo, bez wtyczek typu Flash Player.

```
<video width="640" height="360" controls autoplay muted loop poster="okładka-filmu.jpg">
  ←— Podajemy kilka formatów w razie gdyby stara przeglądarka czegoś nie obsługiwała —→
  <source src="film.mp4" type="video/mp4">
  <source src="film.webm" type="video/webm">
  <p>Twoja przeglądarka nie wspiera wideo w HTML5.</p>
</video>
```

Atrybuty odtwarzaczy:

- `controls` – Dodaje domyślny panel z play/pause/suwakiem/głośnością.
- `autoplay` – Film startuje automatycznie. W nowożytnych przeglądarkach w 99% **zadziała tylko w połączeniu z atrybutem `muted`** (nie wolno straszyć ludzi dźwiękiem bez ich zgody!).
- `muted` – Wyciszenie.
- `loop` – Film pętli się w nieskończoność.
- `poster` – Obraz wstawiany jako miniatura (okładka), zanim klikniemy Play.

Dla tagu `<audio>` składnia jest identyczna (bez width/height). Posiada on atrybuty: controls, autoplay, muted, loop.

```
<audio controls>
  <source src="podcast.mp3" type="audio/mpeg">
  Brak wsparcia audio.
</audio>
```

5.5. Elementy zewnętrzne (embedowanie przez `<iframe>`)

`<iframe>` (Inline Frame) służy do ładowania w naszej stronie innej, oddzielnej strony z internetu (tzw. "strona w stronie"). Najczęstsze zastosowania: filmy z YouTube, mapy Google.

```

<!-- Przykład kodu wyciągniętego prosto z serwisu YouTube (Opcja Udostępnij → Umieść) →
<iframe
  width="560"
  height="315"
  src="https://www.youtube.com/embed/dQw4w9WgXcQ"
  title="YouTube video player"
  frameborder="0"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
  allowfullscreen>
</iframe>
```

- Atrybut `allowfullscreen` pozwala na uruchomienie ramki na pełnym ekranie.

(Zakończenie Części 1)

6. Tabele w HTML

Tabele HTML były historycznie (niesłusznie!) wykorzystywane do budowania układu całych stron (tzw. layout na tabelkach z lat 90). Dzisiaj **absolutnie tego zabramy**. Tabele służą wyłącznie do prezentowania danych tabelarycznych (rozliczenia, cenniki, rozkłady jazdy).

6.1. Podstawowa struktura tabeli

Tabela opiera się na prostym modelu siatki (wiersze i komórki wewnątrz wierszy).

TAG HTML	PEŁNA NAZWA	ROLA I ZNACZENIE
<code><table></code>	Table	Główny kontener otaczający całą tabelę.
<code><tr></code>	Table Row	Wiersz tabeli. Otacza rząderek komórek.
<code><td></code>	Table Data	Zwykła komórka z danymi (Zawsze wewnątrz <code><tr></code>).
<code><th></code>	Table Heading	Komórka nagłówkowa. Domyślnie pogrubiona i wyśrodkowana.

6.2. Semantyczny podział: Nagłówek, Ciało i Stopka tabeli

Aby tabela była w pełni dostępna (aria, czytniki) i łatwiejsza do stylowania w CSS, dane grupujemy w sekcje strukturalne: `<thead>`, `<tbody>`, `<tfoot>`. Przeglądarka potrafi inteligentnie obsługiwać np. wydruk tabeli powtarzając `<thead>` na każdej nowej stronie papieru.

Dodatkowo, `<caption>` pozwala ustawić podpis dla całej tabeli.

```

<table border="1"> <!-- Border 1 jest tu tylko podglądowo, stylujemy to w CSS -->
  <caption>Cennik naszych pakietów (Ryc. 1)</caption>

  <!-- Nagłówek tabeli -->
  <thead>
    <tr>
      <th>Nazwa Pakietu</th>
      <th>Cena miesięczna</th>
```

```

        <th>Dysk twardy</th>
    </tr>
</thead>

<!-- Główne dane tabeli -->
<tbody>
    <tr>
        <td>Basic</td>
        <td>19 PLN</td>
        <td>5 GB</td>
    </tr>
    <tr>
        <td>Pro</td>
        <td>49 PLN</td>
        <td>50 GB</td>
    </tr>
</tbody>

<!-- Stopka tabeli (podsumowania, sumy itp.) -->
<tfoot>
    <tr>
        <td colspan="3">Podane ceny zawierają podatek VAT.</td>
    </tr>
</tfoot>
</table>

```

6.3. Łączenie komórek (colspan i rowspan)

Często chcemy zbudować tabelę, w której jedna komórka jest rozciągnięta poziomo na wiele kolumn lub pionowo na wiele wierszy (podobnie jak w Excelu "Scal i wyródkuj").

- `colspan="n"` (Column Span) - Zastępuje `n` komórek w tym samym wierszu (w prawo). Pamiętaj by z tego wiersza wykasować pozostałe puste komórki!
- `rowspan="n"` (Row Span) - Zastępuje `n` komórek idąc w dół (do kolejnych wierszy). Pamiętaj by z wiersza pod spodem usunąć komórkę z tej kolumny!

```

<table>
  <tr>
    <!-- Komórka rozciągnięta w pionie na 2 wiersze w dół -->
    <td rowspan="2">Kategoria: Elektronika</td>
    <td>Smartfon</td>
  </tr>
  <tr>
    <!-- Zauważ brak pierwszego td w tym wierszu, bo zostało zajęte przez rowspan z góry -->
    <td>Laptop</td>
  </tr>
  <tr>
    <!-- Komórka rozciągnięta na 2 kolumny w poziomie (w prawo) -->
    <td colspan="2" style="text-align: center;">Koniec działu elektroniki</td>
  </tr>
</table>

```

7. Formularze w HTML

Formularze to główne okno na dwustronną komunikację pomiędzy użytkownikiem, a serwerem (logowanie, rejestracje, wiadomości).

7.1. Struktura główna `<form>`

Wszystkie kontrolki formularza zawsze umieszczamy wewnątrz elementu kontenera `<form>`.

```
<form action="/odbior-danych.php" method="POST">

  <!-- Kontrolki wewnątrz -->

</form>
```

Atrybuty kontenera:

- `action` - Adres URL na serwerze (endpoint/endpoint pliku), do którego po wciśnięciu "Wyślij" dane zostaną wysłane. (W środowisku React/SPA action jest często ignorowany).
- `method` - Metoda wysyłki (tzw. metoda HTTP).
 - `GET` - dane dopisują się jawnie do paska adresu URL (np. `?szukaj=buty`). Służy TYLKO do pobierania danych (np. wyszukiwarka na stronie).
 - `POST` - dane ukryte są w ciele żądania (body) HTTP. Używamy jej do wysyłania wrażliwych danych (logowanie, rejestracja, modyfikacje stanu na serwerze).
- `enctype` - Sposób kodowania. Niezbędny jeśli na formularzu przesyłamy *pliki z dysku*. Ustawiamy go na `multipart/form-data`.

7.2. Element `<input>` - Główne wejście

Najpopularniejszy znacznik, którego zachowanie diametralnie się zmienia na podstawie atrybutu `type`. Znacznik samozamykający!

Każdy input w profesjonalnym formularzu powinien posiadać atrybut `name` (Klucz, pod jakim backend zidentyfikuje wartość z tego pola). Np. `imie=Jan, name="imie" → value="Jan"`.

RODZAJE (TYPY) INPUTÓW:

1. Tekst i liczby:

```
<!-- Zwykły krótki tekst -->
<input type="text" name="imie" placeholder="Wpisz imię..." value="Jan" required>

<!-- Hasło (znaki kropka/gwiazdka) -->
<input type="password" name="haslo" required minlength="8">

<!-- Email (automatyczna walidacja znaku @) -->
<input type="email" name="poczta" placeholder="nazwa@domena.pl">

<!-- Liczba (przyciski góra/dół, blokada tekstu alfabetu) -->
<!-- Atrybuty min, max i step ograniczają przedział (np. liczby parzyste od 0 do 100) -->
<input type="number" name="wiek" min="18" max="99" step="1">

<!-- Szukanie (dodaje "X" w przeglądarce by oczyścić pole) -->
<input type="search" name="szukaj_produkow">

<!-- Zakamuflowane pole z danymi (użytkownik go nie widzi, a leci z POST) -->
<input type="hidden" name="id_uzytkownika" value="5412">
```

2. Formaty czasowe i specjalne:

```

<!-- Kalendarz (wybór roku, miesiąca, dnia) -->
<input type="date" name="data_urodzenia">

<!-- Czas (wybór godziny i minuty) -->
<input type="time" name="godzina_wizyty">

<!-- Kalendarz z czasem w jednym elemencie -->
<input type="datetime-local" name="spotkanie">

<!-- Kolorowy picker (wybór koloru paletą HEX) -->
<input type="color" name="ulubiony_kolor" value="#ff0000">

<!-- Wybór pliku z komputera -->
<input type="file" name="cv_pdf" accept=".pdf,.doc,.docx" multiple>

<!-- Suwak dla zakresu wartości z predefiniowanym zakresem -->
<input type="range" name="glosnosc" min="0" max="100" value="50">

```

7.3. Inputy logiczne (Checkboxy i Radia)

Te typy formularzy przyjmują stan "zaznaczony" lub "niezaznaczony". Rządzą się ważnymi prawami atrybutu `name`.

Radio Button (Wybór jednokrotny z wielu) Aby przyciski "radio" grupowały się (czyli by dało się wybrać tylko 1 płeć naraz), muszą posiadać **identyczny atrybut** `name`. Atrybut `value` wysyła wartość na serwer po wciśnięciu danego przycisku.

```

<p>Wybierz płeć:</p>
<input type="radio" id="m" name="plec" value="Mężczyzna"> Kobieta<br>
<input type="radio" id="k" name="plec" value="Kobieta" checked> Mężczyzna<br>

```

Checkbox (Wybór wielokrotny lub Zgody) Służą do wyboru opcji niezależnych od siebie (lub potwierdzania np. "Akceptuję regulamin").

```

<p>Wybierz dodatki do pizzy:</p>
<input type="checkbox" name="dodatki" value="ser" checked> Dodatkowy Ser<br>
<input type="checkbox" name="dodatki" value="bekon"> Bekon<br>

<input type="checkbox" name="zgoda" required> Akceptuję Regulamin

```

7.4. Znacznik powiązania - `<label>`

Label czyli Etykieta to krytyczny znacznik Dostępności i ergonomii. Jeśli przypiszemy Label do Inputa, kliknięcie myszką na słowo "Mężczyzna" zaznaczy pole Radio, co drastycznie ułatwia życie użytkownikom telefonów z grubymi palcami. Oprócz tego pozwala czytelnikowi głosowemu rozpoznać, do czego służy input.

Robimy to poprzez atrybut `for` przypisany do `id` inputa:

```

<label for="haslo_id">Wpisz tajne hasło:</label>
<input type="password" id="haslo_id" name="haslo">

```

7.5. Pola wieloliniowe (`<textarea>`)

Kiedy potrzebujemy napisać referat lub wiadomość kontaktową, input text to za mało, nie obsługuje on bowiem klawisza "Enter" w sobie. Wykorzystujemy `textarea`.

```
<label for="wiad">Twoja wiadomość:</label>
<textarea id="wiad" name="wiadomosc" rows="5" cols="40" placeholder="Pisz tutaj..."></textarea>
```

7.6. Listy rozwijane (Combo box / Select)

Klasyczny dropdown z opcjami. Atrybut `multiple` pozwala wybierać wiele z klawiszem Ctrl, a `selected` na danej `<option>` sprawia że pole domyślnie ustawia się na ten wybór.

```
<label for="auto">Wybierz markę auta:</label>
<select id="auto" name="marka">
  ←— Opcjonalne optgroup kategoryzuje listę →
  <optgroup label="Niemieckie">
    <option value="audi">Audi</option>
    <option value="bmw" selected>BMW</option>
  </optgroup>
  <optgroup label="Francuskie">
    <option value="peugeot">Peugeot</option>
    <option value="renault">Renault</option>
  </optgroup>
</select>
```

7.7. Przyciski i Zatwierdzanie (`<button>`)

Aby odesłać zawartość tagu `<form>` do akcji (serwera), wystarczy jeden przycisk.

- `<button type="submit">` - (Domyślne zachowanie wewnątrz forma!) Po wciśnięciu tego przycisku formularz jest zamykany (zatwierdzany, walidowany, wysyłany POSTem). Użycie tagu button zamiast `<input type="submit">` pozwala na wkładanie ikonki do środka przycisku.
- `<button type="button">` - Martwy przycisk. Nic nie robi z formularzem, przydaje się gdy chcemy zapiąć własny kod pod JavaScript (np. kalkulator).
- `<button type="reset">` - Zeruje zawartość pól formularza.

7.8. Walidacja Formularzy (HTML5)

HTML5 potrafi natywnie rzucać komunikaty "Wypełnij to pole" lub "Adres email zawiera błąd" z blokadą wysyłki, bez grama JavaScriptu!

Atrybuty walidacyjne (wkładane prosto do tagu `<input>`):

- `required` - Oznacza pole jako obowiązkowe (musi mieć znaki).
- `minlength="x" maxlength="y"` - Limity długości znaków dla haseł i opisów.
- `min="x" max="y"` - Limit numeryczny (albo datowy) - blokuje dziwne daty z przyszłości czy zły wiek.
- `pattern="[0-9]{2}-[0-9]{3}"` - Wprowadza silnik wyrażeń regularnych (RegEx) sprawdzający dopasowanie maski. Ten wyżej np. zatwierdzi jedynie tekst w formacie kodu pocztowego z polski XX-XXX.

Aby anulować odgórną walidację przeglądarki (bo piszemy własną np. w React) - należy dodać na sam kontener atrybut logiczny `<form novalidate>`.

8. Elementy semantyczne HTML5

8.1. Czym jest Semantyka

Semantyka to opisywanie **znaczenia** danego fragmentu strony zamiast opisywania jego wyglądu. HTML przed wersją 5 opierał się na setkach anonimowych pudełek (`<div id="header">` , `<div class="footer">`). Dla robota Google było to pozbawione sensu i znaczenia, co wpływało źle na pozycjonowanie.

Dlatego wprowadzono zbiór predefiniowanych "pudełek z imieniem", które działają dokładnie tak samo jak zwykły obojętny `<div>`, ale informują roboty Google i Czytniki Ekranowe: "Hej, ja jestem nawigacją!".

8.2. Główny szkielet strony

- `<header>` - Przeważnie na samej górze. Zawiera logo, nawigację strony lub nagłówki na górze artykułu.
- `<nav>` - Blok z głównymi linkami nawigacyjnymi (często wewnątrz `<header>`). Używamy TYLKO dla głównych menu, a nie pojedynczego linku z puszki.
- `<main>` - Dominująca i główna unikalna część dla danego adresu podstrony. Na stronie powinien być **tylko jeden taki element**.
- `<footer>` - Stopka strony na samym dole. Noty prawne, linki kontaktowe, regulamin, prawa autorskie.
- `<aside>` - Panel poboczny (tzw. sidebar). Są to np. banery reklamowe z boku, moduły z najpopularniejszymi tagami, które powtarzają się wszędzie a nie są głównym mięsem serwisu. Omijane przez czytniki.

8.3. Komponenty Treści (Article i Section)

Wewnątrz znacznika `<main>` stronę musimy logicznie podzielić na kawałki.

- `<article>` - Reprezentuje samodzielną, kompletną, całkowicie samowystarczającą część kompozycji. Przykłady: pojedynczy wpis na blogu, wiadomość na forum, nowina na portalu, recenzja użytkownika. "Samowystarczalność" określamy tak: jeśli wrzucę `<article>` w ramkę i wstawię na całkiem inną domenę - czy będzie miał sens dla kogoś kto to czyta? Tak!
- `<section>` - Ogólny blok grupujący treści danego tematu (często wymaga wewnątrz nagłówka powiązanego z `<hX>`). Reprezentuje zwykły dział strony, np. kategoria powiązanych newsów, albo sekcja kontakt z mapą na dole.

```
<body>

  <header>

    

    <nav>

      <ul>

        <li><a href="/">Strona Główna</a></li>

        <li><a href="/artykuly">Artykuły</a></li>

      </ul>

    </nav>

  </header>

  <main>

    <section class="ostatnie-news">

      <h2>Najnowsze doniesienia z IT</h2>

      <article>

        <header>

          <h3>Wydanie HTML5 oficjalne</h3>

          <p>Autor: ABC | <time datetime="2014-10-28">28.10.2014</time></p>

        </header>

        <p>Nareszcie nowa wersja...</p>

        <footer>Tagi: html, frontend</footer>

      </article>

      <article>

        <h3>Drugi news o Apple</h3>
```



```

        <p>Krótki news wewnątrz tej samej sekcji.</p>
    </article>
</section>
</main>

<aside>
    <h3>Polecani Partnerzy</h3>
    <p>Reklama.</p>
</aside>

<footer>
    <p>&copy; Wszelkie Prawa zastrzeżone.</p>
</footer>
</body>

```

8.4. Interaktywne Elementy Semantyczne (Details i Summary)

Znacznik tworzący "rozwijak / akordeon" w natywnym HTML bez krztyny JS (niezwykle użyteczny w tworzeniu modułów pytań Q&A).

```

<details>
    <summary>Pytanie: Dlaczego kodowanie UTF-8 jest polecane?</summary>
    <p>Odpowiedź: Ponieważ obsługuje polskie znaki diakrytyczne (ę, ó, ą) oraz miliardy znaczków, emoji itp, czego nie potrafi np.
    latin2.</p>
</details>


<details open>
    <summary>Jaka jest pojemność pendrive'a?</summary>
    <p>16 GB.</p>
</details>

```

8.5. Czas, Postęp, Okna Modalne

- `<time datetime="2026-06-06">Dzisiaj</time>` - mówi robotom Google "Słowo dzisiaj w tym tagu oznacza fizycznie ten i ten dzień o danej porze GMT+1".
- `<progress max="100" value="70"></progress>` - Rysuje natywny pasek ładowania postępu pliku.
- `<meter min="0" max="10" low="3" high="8" optimum="5" value="9"></meter>` - Rysuje wskaźnik, super na miejsce w dysku. Gdy wejdzie powyżej "high" - zaświeci na czerwono.
- `<dialog id="modal_id">` - Znacznik okna wyskakującego (Pop-upa). Sam nakłada ciemne tło przy uruchomieniu z JS za pomocą `document.getElementById('modal_id').showModal()`.

9. Atrybuty globalne i Dostępność (WAI-ARIA)

Zapewnienie dostępności (Accessibility, w skrócie A11y) to obowiązkowy element w nowoczesnym tworzeniu stron internetowych. Dostępność gwarantuje, że ze strony będą mogli korzystać wszyscy użytkownicy, w tym osoby z niepełnosprawnościami, np. korzystające z czytników ekranowych czy nawigujące wyłącznie za pomocą klawiatury.

9.1. Atrybuty WAI-ARIA

Zestaw atrybutów WAI-ARIA (Web Accessibility Initiative - Accessible Rich Internet Applications) służy do przekazywania dodatkowych informacji o strukturze i zachowaniu elementów interfejsu czytnikom ekranowym i innym technologiom asystującym. Stosuje się je najczęściej tam, gdzie brakuje wbudowanych tagów semantycznych lub gdy tworzymy niestandardowe komponenty w JavaScript.

- `aria-label="Zamknij"` – Deklaruje etykietę tekstową dla elementu, z pominięciem wizualnego tekstu. Zawsze używana, gdy element (np. przycisk) posiada wyłącznie ikonę bez opisu.
- `aria-hidden="true"` – Informuje technologie asystujące, że dany element i jego potomkowie powinni zostać całkowicie zignorowani przy odczytywaniu strony (stosowane np. dla dekoracyjnych ikon).
- `aria-expanded="true/false"` – Informuje, czy interaktywny kontener (taki jak rozwijane menu lub akordeon) jest obecnie rozwinięty czy zwinięty.
- `role="button"` – Przekazuje technologii asystującej informację o semantycznej roli elementu (np. informuje, że zwykły `<div>` pełni rolę klikalnego przycisku). Należy go łączyć z implementacją obsługi zdarzeń klawiatury (Spacja, Enter).
- `aria-live="polite"` – Deklaruje, że zawartość danego elementu może być dynamicznie aktualizowana, a zmiany te powinny być anonosowane przez czytnik w momencie, gdy użytkownik nie wykonuje innych czynności. Często stosowane przy tzw. komunikatach "Toast" (np. "Wiadomość została wysłana").

9.2. Własne atrybuty `data-*`

Jeśli specyfikacja HTML nie oferuje dedykowanego atrybutu do przechowania specyficznych informacji programistycznych, można zastosować standard `data-*`. Pozwala on na zagnieżdżanie w drzewie DOM dowolnych niestandardowych metadanych, które zachowują ważność syntaktyczną HTML5 i są bezpieczne w odczycie po stronie kodu JavaScript.

```
<div class="product-card" data-id="1024" data-category="electronics" data-discount="15">
  <p>Dell XPS 15</p>
</div>
```

Każdy element DOM implementuje właściwość `dataset`, będącą interfejsem odczytu tychże atrybutów.

```
const element = document.querySelector('.product-card');
console.log(element.dataset.id); // Wypisze "1024"
console.log(element.dataset.category); // Wypisze "electronics"
```

10. Wprowadzenie do JSX (HTML w React)

JSX (JavaScript XML) to rozszerzenie składni języka JavaScript, szeroko stosowane w środowiskach opartych o bibliotekę React. Umożliwia pisanie deklaratywnych struktur interfejsu za pomocą tagów HTML bezpośrednio wewnątrz logiki programu. Po stronie procesu budowania projektu (przy wykorzystaniu transpilatorów, np. Babel), kod JSX jest konwertowany do klasycznych wywołań API `React.createElement(...)`. Przeglądarka internetowa wykonuje wynikowy kod JavaScript i ostatecznie manipuluje standardowym drzewem DOM.

10.1. Zasadnicze różnice między specyfikacją HTML a JSX

Mimo że składnia wygląda bardzo podobnie, JSX rygorystycznie opiera się na specyfikacji języka JavaScript. Z tego powodu wiele standardowych atrybutów zyskało nowe nazewnictwo (zastosowano notację CamelCase), a niektóre reguły zostały uściśnione, aby wyeliminować potencjalne błędy i unikać kolizji ze słowami kluczowymi języka JS.

ATRYBUT / CECH W HTML	ODPOWIEDNIK W JSX	ZMIANA Z UWAGI NA SPECYFIKĘ
<code>class="btn primary"</code>	<code>className="btn primary"</code>	Słowo <code>class</code> w JavaScript stanowi zarezerwowane słowo kluczowe służące do deklarowania obiektowych klas ES6.
<code>for="inputID"</code>	<code>htmlFor="inputID"</code>	Słowo <code>for</code> jest również zarezerwowane w JS do tworzenia pętli iteracyjnych.
<code>onClick="run()"</code>	<code>onClick={run}</code>	Zdarzenia nasłuchujące w React wymagają zapisu w formacie camelCase oraz przekazania referencji do funkcji zamiast łańcucha tekstowego.
<code>onChange</code> / <code>onsubmit</code>	<code>onChange</code> / <code>onSubmit</code>	Analogiczna zasada zapisu dla dowolnych zdarzeń DOM.
<code>style="color: red; padding-top: 10px;"</code>	<code>style={{ color: 'red', paddingTop: '10px' }}</code>	Atrybut stylu w JSX pobiera obiekt w formacie JavaScript, a nie łańcuch tekstowy. Ze względu na składnię JS z użyciem dywizów (np. <code>padding-top</code>), wymuszone jest tu nazewnictwo camelCase.
<code></code> (zostawione otwarte)	<code></code>	Krytyczne: W JSX bezwzględnie każdy element tagowy niezawierający zawartości wewnątrz musi zamykać się sam przez ukośnik. W przeciwnym razie kompilator zgłosi błąd składni.
<code><!-- Komentarz --></code>	<code>{/* Komentarz */}</code>	Jako, że kod wewnątrz musi podlegać regułom JavaScript, używane są tu komentarze blokowe ze znakiem obłożenia w ewaluację klamrową.
<code>tabindex="1"</code>	<code>tabIndex="1"</code>	Obowiązuje konwencja camelCase.

Przykład komponentu formularza w ujęciu Reactowym:

```
function LoginForm() {
  return (
    <form className="login-form" onSubmit={handleLogin}>
      <label htmlFor="usernameInput">Login:</label>
      <input
        id="usernameInput"
        type="text"
        className="input-field"
        autoFocus
      />
      {/* Przycisk */}
      <button type="submit" style={{ backgroundColor: '#0056b3', color: '#ffffff' }}>
        Zaloguj
      </button>
    </form>
  );
}
```

10.2. Osadzanie dynamicznych wyrażeń JavaScript w JSX

Kluczową zaletą JSX jest zdolność natywnego i bezpośredniego wstrzykiwania kodu logicznego pomiędzy bloki układu HTML. Za pomocą nawiasów klamrowych `{ }` informujemy kompilator, że przekazana zawartość ma zostać odczytana jako poprawny blok ewaluacyjny JavaScript, a jego zwrócony wynik wpisany w to miejsce widoku.

```
function WelcomePanel() {
  const userName = "Kacper";
  const currentYear = new Date().getFullYear();
  const experienceYears = 5;

  return (
```

```

<div>
  {/* Wyrenderowanie stałej tekstowej zdefiniowanej w ciele funkcji */}
  <h1>Witaj, {userName}!</h1>

  {/* Obliczenie matematyczne wykonane podczas renderowania wirtualnego DOM */}
  <p>Szacunkowa łączna liczba projektów: {experienceYears * 12}</p>

  {/* Użycie zwróconej wartości wywołanej w locie struktury JS */}
  <p>Prawa autorskie dla roku {currentYear}</p>
</div>
);
}

```

Uwaga techniczna: Bloki `{ }` obsługują wyłącznie tzw. wyrażenia (expressions), z których jednoznacznie w locie powraca pojedyncza wartość. Nie należy umieszczać tu złożonych instrukcji sterujących wykonaniem jak pętle `for` czy struktury decyzyjne `if/else`.

10.3. Renderowanie list danych i zarządzanie kluczami (`key`)

Biblioteka nie implementuje gotowej dyrektywy powtarzania. Tworzenie całych kolekcji elementów na podstawie struktury z bazy danych rozwiązuje się poprzez funkcję iteracji wyższego rzędu – `Array.map()`. Transformuje ona zbiór danych na tablicę powiązanych elementów tagów HTML, które są na końcu rozwijane w rodzicu kontenerze.

```

const userList = [
  { id: 1001, name: "Kacper" },
  { id: 1002, name: "Anna" }
];

function EmployeeDirectory() {
  return (
    <ul>
      {userList.map((user) => (
        <li key={user.id}>
          Profil pracownika: {user.name}
        </li>
      ))}
    </ul>
  );
}

```

Atrybut identyfikacji `key`: Podczas dynamicznego generowania tablicy komponentów w drzewie, algorytm React (*Virtual DOM Diffing*) kategorycznie wymaga sprecyzowania dla każdego węzła jednoznacznego klucza w atrybucie `key`. Służy to stabilności widoków i potężnym optymalizacjom renderingu przy procesie godzenia (Reconciliation). Dzięki temu silnik trafnie potrafi zmierzyć m.in. usunięcie obiektu 1001 lub aktualizację wyłącznie w 1002 bez wymuszania odświeżenia graficznego całej tabeli. Należy wystrzegać się wstrzykiwania do tego miejsca wartości iteracyjnego `index`, który potrafi destabilizować formularze z własnym stanem podczas np. rotowania widoku sortowania. Należy posługiwać się identyfikatorami pochodzącymi natywnie z warstwy zapisu informacji (bazy danych SQL/NoSQL).

10.4. Fragmenty (Zasada jednego rodzica)

Działania każdej definicji zwracanej z komponentu polegają na utworzeniu w pamięci drzewa węzłów. Kompilator nie zezwala, aby podstawa elementu kończyła się wieloma głównymi zjawiskami na zerowym poziomie rodzica bez zamknięcia we wspólnym kontenerze. Błąd poniżej jest bardzo częsty.

```
// Antywzorzec - Błąd kompilatora. Węzły stoją obok siebie i nie zawiązają korzenia.
function ArticleHeader() {
  return (
    <h1>Zalety testowania</h1>
    <p>W dzisiejszych czasach...</p>
  )
}
```

By uniknąć wprowadzania zbędnego kontenera DIV w strukturę układu HTML i nadmiernego pompowania zagnieżdżeń (zjawisko nazywane powszechnie "div soup"), w ekosystemie stosuje się technologię abstrakcyjnych Fragmentów (skrót do `React.Fragment`).

```
function ArticleHeader() {
  return (
    <>
      <h1>Zalety testowania</h1>
      <p>W dzisiejszych czasach...</p>
    </>
  )
}
```

10.5. Atrybuty Boolowskie

W przypadku atrybutów, które obsługują stany logiki z prawdy w fałsz, zapisanie wartości nazwy samej w sobie bez definicji w klamrach konwertuje automatycznie domyślne przydzielenie wartości `true`. Poniższy zapis obu pól deklaruje brak aktywności bloku wejściowego na równym statusie.

```
<input type="text" disabled={true} />
<input type="text" disabled />

{/* Powiązanie z deklaratywnym odniesieniem zmiennej lokalnej. */}
<button disabled={itemsInCart === 0}>Podsumowanie</button>
```

10.6. Warunkowe wstrzykiwanie do drzewa JSX (Renderowanie Warunkowe)

Zjawisko chowania i wyświetlania bloków decyzyjnych wymusza brak używania bezpośredniej instrukcji warunkowej `if/else` pomiędzy tagami (nieдозwolone). Deweloperzy rozwiązują to przez stosowanie wysoce operatywnego *Operatora Warunkowego* (Ternary Operator) w wariancie logicznym pełnym lub wykorzystaniu zjawiska ewaluacji Short-Circuit AND (`&&`) dla zjawisk zerwania binarnego.

```
function NotificationsPanel({ isAuthenticated, unreadCount }) {
  return (
    <nav>
      {/* Wykorzystanie logiki Short-Circuit. Jeśli warunek lewej strony ewaluuje się pozytywnie, wyświetla ułamek komponentu po stronie prawej. W sytuacji negatywnej struktura elementu span nigdy nie zostaje dopisana na węzeł DOM. Zabezpiecza to kod przed ukrywaniem wyłącznie poprzez regułę widoczności z warstwy prezentacyjnej (display: none). */}
      {unreadCount > 0 && <span className="badge">Powiadomienia: {unreadCount}</span>}

      {/* Operator Ternary, wymusza precyzyjne ścieżki i powroty z decyzji w dwóch wymiarach. */}
      {isAuthenticated ? (
        <button>Panel użytkownika</button>
      ) : (
```

```

        <button>Zaloguj się</button>
      )}
    </nav>
  );
}

```

Mechanizm reaguje na wyrażenie bezpośredniego wymuszenia ukrycia interfejsu (Early return) po przez odesłanie u podstawy `return null;`. Taki zabieg nie produkuje w architekturze DOM żadnego elementu wyjściowego z danej procedury.

10.7. Niekontrolowane i kontrolowane formularze

Zasadniczo, tradycyjny silnik HTML przechowuje wszelkie zapisane we wnękach wejściowych informacje we własnym lokalnym systemie zachowując pełne odzwierciedlenie tego interfejsu (formularze niekontrolowane). Ekosystem programowy modyfikuje to zjawisko promując powszechnie Formularze Kontrolowane. Architektura ta zakłada, że całkowite zarządzanie faktycznym stanem elementu kontrolki posiada logika wewnętrzna instancji komponentu we współdzielonej metodzie zapisu RAM, do momentu wystania żądania sieciowego.

Każdorazowe dodanie pojedynczego znaku w widoku pola wymusza synchronizację przez następujące zdarzenie modyfikujące. Mechanizm w ten sposób pozwala np. na zablokowanie wejścia lub nałożenia precyzyjnych transformacji jeszcze przed odbiciem wartości wizualnej z renderem.

```

import { useState } from 'react';

function SubscriptionForm() {
  const [emailAddress, setEmailAddress] = useState("");

  const handleSubmit = (event) => {
    // Obiekt SyntheticEvent reprezentuje mechanizm zdarzeniowy wejścia, wyłączenie na nim funkcji Default zatrzyma archaiczne
    // odświeżenie karty przeglądarki wywodzące się z wdrożeń formularzy historycznych HTML.
    event.preventDefault();
    console.log("Próba zapisu dla adresu: " + emailAddress);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Adres E-mail:
        <input
          type="email"
          value={emailAddress}
          onChange={ (e) => setEmailAddress(e.target.value) }
          required
        />
      </label>
      <button type="submit">Dołącz do Subskrypcji</button>
    </form>
  );
}

```

Dla typologii elementów checkbox wymusza się zablokowanie atrybutu odczytu jako odwołania do wartości stanowej domyślnej `event.target.checked` ze względu na jej logiczną specyfikację powrotu z ujęcia binarnych decyzji wyboru pola wielokrotnego.

```
const [termsAccepted, setTermsAccepted] = useState(false);
// ...
☒
```

10.8. Zastosowanie modularne

Krytycznym czynnikiem spopularyzowania narzędzi tej klasy stała się zasada ponownego wykorzystywania niezależnych jednostek wizualnych oraz logicznych bez nadpisywania spetryfikowanych bloków HTML w wielomilionowych dokumentach archiwizowanych po wielokroć dla spersonalizowanych rzutów bazy danych. Wzorzec projektowy określa się mianem Deklaratywnego Zarysu Systemów Interfejsu (Declarative User Interface). Komponenty nie stają się niczym innym, niż udokumentowanym elementem wykonawczym przyjmującym ujęcie dynamicznych parametryzacji w bloku definiowanych powszechnie pod nazwą **Props** od atrybutów właściwości wejścia danych. Powielając instancję jednego modułu renderowania graficznego można wytworzyć bardzo spójną listę wielotysięcznych obiektów.

11. Wprowadzenie do CSS (Cascading Style Sheets)

Kaskadowe Arkusze Stylów (CSS) stanowią integralną część technologii webowych, odpowiadając za warstwę prezentacyjną aplikacji. O ile HTML nadaje strukturę informacjom, CSS zajmuje się interpretacją reguł wizualnych, formatowaniem i pozycjonowaniem obiektów dla renderującego silnika graficznego. Oddzielenie tych dwóch domen umożliwia pełną modularyzację projektu.

11.1. Metody dołączania stylów CSS

Silnik implementujący posiada trzy ścieżki weryfikacji i nakładania reguł graficznych na obiekt wyjściowy.

1. Style Zewnętrzne (External CSS) Rekomendowane podejście w profesjonalnych wdrożeniach produkcyjnych. Utrzymuje pełną izolację logiki prezentacji od struktury oraz pozwala wykorzystywać zaawansowane bufor optymalizacji po stronie pamięci podręcznej serwerów (caching). Arkusz dołączany jest za pomocą tagu łączącego zdefiniowanego w sekcji głowy dokumentu (**<head>**).

```
<link rel="stylesheet" href="styles/main.css">
```

2. Style Wewnętrzne (Internal CSS) Wykorzystywane dla niewielkich fragmentów dokumentów uwarunkowanych bardzo rzadką częstotliwością edycji i wykluczających obciążenia zewnętrznymi plikami z uwagi na czas wczytywania (np. w systemach szablonów mailingowych).

```
<head>
  <style>
    body { font-family: sans-serif; }
  </style>
</head>
```

3. Style Liniowe (Inline CSS) Wysoce specyficzna iniekcja parametru do obiektu DOM, zdefiniowana na poziomie atrybutu elementu. Metoda kategorycznie ograniczana z uwagi na łamanie standardów i nadpisywanie niemal wszystkich kaskadowych dziedziczeń z wag hierarchii (z uwagi na ekstremalną siłę priorytetową, specyficzność). Powszechnie stosowana przez frameworki JS dla atrybutów dynamicznie generowanych przy każdym cyklu wirtualnego interfejsu (np. postęp paska ładowania).

```
<div style="color: blue; padding: 10px;">Tekst</div>
```

11.2. Anatomia Reguły CSS i Kaskada

Pojedyncza reguła określa logiczne odniesienie do elementu strukturalnego za pomocą wskazanego Selektora, wprowadzając zbiór właściwości prezentacyjnych zwieńczonych znakami zamknięcia (średnik).

```
/* Zbiór nazywany powszechnie deklaracją */
h1 {
  color: red; /* Właściwość : Wartość; */
  font-size: 24px;
}
```

Kaskadowość (The Cascade) Rdzeń interpretacyjny języka CSS zależy od zaawansowanego algorytmu obliczeniowego nazywanego kaskadą, który ocenia sposób alokacji nakładających się i wykluczających właściwości celujących w ten sam obiekt wizualny. Mechanizm interpretuje instrukcje w porządku źródłowym – deklaracje wpisane później (np. na dole pliku) przy równej specyfice wagi selektorów mogą nadpisać poprzednie reguły graficzne.

11.3. Wzorzec Dziedziczenia Właściwości (Inheritance)

Część zadeklarowanych parametrów podlega natywnemu dziedziczeniu. Skonfigurowanie właściwości na głównym rodzicu w węźle dokumentacyjnym rzutuje w głąb hierarchii drzewa DOM. Do właściwości dziedzicznych zaliczają się m.in. zmienne powiązane z modułem typografii (kolor znaków, stylizacje liter, wielkość fontu, wyrównanie). Natomiast parametry operujące architekturą rozmiaru (wymiar, marginesy) nigdy nie podlegają rozszerzeniu hierarchicznemu. Właściwość dziedziczną można również w przypadku komplikacji wymusić słowem kluczowym `inherit`.

12. Selektory CSS (Zarządzanie Zasięgiem)

Wymogiem modyfikowania właściwości interfejsu jest zdolność precyzyjnego filtrowania strukturalnych zbiorów znaczników. CSS operuje selektorami jako logicznymi uchwytami, z których każdy posiada ustaloną wagę priorytetową (tzw. Specificity).

12.1. Podstawowe selektory

```
/* Selektor Typu (Znacznika) - Najniższa specyficzność wagowa w procesie */
p {
  line-height: 1.5;
}

/* Selektor Klasy - Opatrzony prefixem (.). Wskazuje na element HTML wykorzystujący przypisany atrybut class="nazwa". Uniwersalny,
preferowany standard stylowania modularnego. */
.primary-button {
  background-color: blue;
}

/* Selektor Identyfikatora - Opatrzony prefixem (#). Element musi nosić unikalny w skali dokumentu atrybut id="nazwa". Wysoka
specyficzność utrudnia nadpisywanie w przyszłości, co często odradza się w dojrzałych projektach. */
#main-navigation {
  width: 100%;
}

/* Selektor Uniwersalny (*) - Obejmuje absolutnie wszystkie znaczniki zarejestrowane po stronie widoku graficznego elementu okna.
Często stosowany do resetów globalnych obramowań (box-sizing). */
* {
  margin: 0;
}
```


12.2. Kombinatory Selektorów

Wymuszają filtrowanie ze względu na powiązania między konkretnymi gałęziami węzłów HTML (pokrewieństwo, dziedziczenie w drzewie graficznym).

```
/* Kombinator Potomka (Spacja) - Znajdzie wszystkie <a>, które znajdują się w jakimkolwiek miejscu wewnątrz struktury <nav> (nawet
wysoce zagnieżdżone warstwy w głębi) */
nav a { color: white; }

/* Kombinator Dziecka Bezpośredniego (>) - Znajdzie tylko <li> będące bezpośrednio podrzędne wobec ul (warstwa 1 w dół),
wykluczając wnuków znajdujących się w dalszych zagnieżdżeniach sub-list */
ul > li { padding-left: 20px; }

/* Kombinator Rodzeństwa Zbliżonego (+) - Celuje wyłącznie w pierwszy znacznik <p>, który znajduje się bezpośrednio obok (po linii
siostrzanej) znacznika h1 */
h1 + p { font-weight: bold; }
```

12.3. Pseudoklasy (Zmiany Stanowe Interakcji)

Rejestrują modyfikacje własności wygenerowane z aktywności kursora, klawiatury użytkownika, logiki okienek systemowych albo pozycji indeksowej węzła DOM względem węzłów bliźniaczych. Wykorzystują notację z użyciem jednego znaku dwukropka `:`.

ZAPIS STRUKTURALNY	PRZEZNACZENIE OPERACYJNE W LOGICE SYSTEMU ZDARZEŃ I HIERARCHII
<code>:hover</code>	Załącza właściwości tylko podczas czasu przebywania wskaźnika systemowego, myszy ponad przypisanym elementem graficznym.
<code>:focus</code>	Niezbędna właściwość wspierania dostępności - narzuca style dla elementu (najczęściej inputu), z którym klient aktualnie dokonuje interakcji wejściowej na systemowej kontrolce lub gdy dotarł do kontenera z klawiatury przyciskiem z indeksowaniem elementów.
<code>:nth-child(n)</code>	Umożliwia włączenie struktury logicznych pętli i algebry z rzutu DOM. Użycie <code>li:nth-child(odd)</code> pokoloruje odmiennie wyłącznie te rzędy listy, które noszą matematyczne wskaźniki elementów po liczbach nieparzystych. Wysoce poszukiwane we wdrażaniach odczytu tablic korporacyjnych.
<code>:first-child</code> / <code>:last-child</code>	Filtruje węzeł wyłącznie wówczas, gdy dany element otwiera blokowy obszar rodzeństwa na pierwszym slocie, bądź ostatecznym indeksowym, skrajnym bloku (ostatnim z listy braci w danym korzeniu rodzica).
<code>:not(.disabled)</code>	Umożliwia zablokowanie rzutu parametrów dla części elementu selektora posiadającej w zagnieżdżeniu inną z góry założoną klasyfikację. Na przykład nie załączy podświetlenia w hover dla instancji, która jest wyłączona (odpięta flagowo w klasyfikacji HTML).

12.4. Pseudoelementy (Tworzenie Węzłów z Powietrza)

Składnia z podwójnym dwukropkiem `::` zadeklarowana we współczesnym standardzie językowym informuje silnik przeglądarki o wygenerowaniu zjawiska niematerialnego od strony struktury znacznikowej binarnego HTML.

Do najpopularniejszych zalicza się pseudoelementy `::before` oraz `::after`. Funkcjonują one jako syntetycznie wtłaczane kontenery wewnątrzkanałowe na element selekcjonowany i powszechnie wymuszają właściwość content do działania rzutu obiektowego. Wykorzystywane głównie dla zjawisk upiększeń i narzutowania wizualizacji dodatków, które mogłyby utrudniać logiczny proces pozycjonowania dokumentu z warstwy plików znaczników.

```
.blockquote::before {
  /* Parametr pustego content jest konieczny w instancji, w innym rzucie przeglądarka odrzuci wymuszony render tego wirtualnego
poła */
  content: "";
  display: block;
  width: 10px;
  height: 100%;
```

```
background-color: blue;
```

```
}
```

12.5. Waga Selektorów i Specyficzność (The Specificity Wars)

Podczas pisania arkuszy stylów, system bardzo często popada w logiczne konflikty wynikające z deklarowania zróżnicowanych wariantów stylów dla tego samego uchwytu w DOM na różnym poziomie wag i zasięgu odwołań kodu źródłowego. Przeglądarka implementuje wirtualny przelicznik potęg i sił rzutu by oszacować, jaka modyfikacja uzyska ostateczny wpływ i wejdzie do odczytu ekranowego.

Algorytm nalicza abstrakcyjne punkty dla każdego selektora uwarunkowując potęgę przydziału:

1. Słowo kluczowe `!important` w zdefiniowanej właściwości posiada wagę krytyczną, nieskończoną. Użytkowanie jest odradzane, doprowadza wielokrotnie do nieodwracalnego i bezpowrotnego zaburzenia architektury projektu i zdolności skalowania po re-faktoryzacji widoku (przebija wszystko z rzutu).
2. Style atrybutowe (Inline CSS – w wierszu `style=""`) otrzymują potęgę rzędu `1000`.
3. Identyfikatory (ID - `#`) dają moc rzędu `100` punktów dla wagi odwołania. Za ledwie jedną klasę nie nadpisze przypisanej mu operacji na rzucie.
4. Klasy i pseudoklasy oraz atrybuty dziedziczące, rzutują siłę systemową wycenianą w `10` jednostkach.
5. Typologię i tagi (`div`, `p`, pseudo-elementy) uwarunkowano standardem o masie na poziomie za ledwie `1` punktu mocy kaskadowej hierarchii dla całego zespołu silnika.

13. Jednostki i Wartości CSS

Wymiarowanie układu wizualnego na potrzeby sieci nie pozwala na operowanie wyłącznie stałymi jednostkami fizycznymi, do których przywykły systemy DTP (Desktop Publishing) przeznaczone dla rynku poligraficznego i prasowego. Ze względu na zróżnicowanie technologiczne urządzeń końcowych, system wymaga wprowadzania zjawisk odniesienia uwarunkowanych z relatywnych skal w architekturze DOM.

13.1. Jednostki Absolutne

Traktowane są we współczesnych warunkach z niechęcią w kontekście responsywnego dostosowania szkieletu dokumentu. Odwołują się do fizycznych zjawisk statycznych uwarunkowanych w silniku bazowym.

- `px` (Piksele) – Powszechnie przyjmuje się 1 px za ułamek wielkości 1/96 cala wyświetlacza systemowego okna, zachowując z reguły ramy wymiarowania absolutnego dla wektorowych definicji struktury we wczesnym wdrożeniu.
- `cm`, `mm`, `pt` – Historyczny zbiór odniesień z poligrafii (centymetr, punkt drukarski), wykorzystywany jedynie w przypadku generacji dokumentacji ukierunkowanej na zjawisko podglądu z dyspozytora wydruku dla arkuszy do `@media print`.

13.2. Jednostki Względne (Relatywne)

Stanowią architektoniczny fundament powszechnie zalecanej koncepcji zjawisk dostosowawczych. Przeliczają wewnętrzną dynamikę na podstawie kontekstu, w którym funkcjonuje konkretny element.

JEDNOSTKA	ARCHITEKTURA ODNIESIENIA Z DOM	ZACHOWANIE I PRZEZNACZENIE W MECHANIZMACH ADAPTACYJNYCH
% (Procenty)	Rozmiar rodzica	Określa odsetek parametru zależny z reguły bazowej rodzica. Jeśli nałożono <code>width: 500px</code> w bezpośrednim zbiorniku przodka, wezwanie do użycia 50% wewnątrz wymusza na wariancie wygenerowanie rzutu 250px dla elementu potomnego. Fundamentalne podejście w konstruowaniu tabel i elastycznych obszarów.
vw	Viewport Width	Szerokość okna z ekranu klienta w całościowym rzucie przeglądarkowym. Ignoruje restrykcje z zagnieżdżenia, <code>100vw</code> odwołuje do krawędzi okien ze stuprocentową pewnością rozszerzenia z wdrożeń dla np. nawigacji bocznej pełnoekranowej.
vh	Viewport Height	Wysokość okna widoku, doskonała dla obrazów bazowych we frakcjach czołowych z dopasowaniem powrotów np. <code>height: 100vh</code> do generacji obszarów tła pod powłoki hero-section bez wykraczania za margines pionowy.
em	Oparty o Typografię Przodka	Mnożnik uwarunkowany bieżącą wartością <code>font-size</code> z reguły włączonej hierarchii drzewa DOM. Idealnie dopasowuje elementy wypełnienia (padding wewnątrz kontrolek), które skalują swobodnie na podstawie proporcji nadanej czcionce. Jego użycie dla czcionek stwarza często ryzyko rekursywnego zwiększenia (Compounding) przy braku zachowania szczególnej ostrożności w głębokich hierarchiach pętli nawigacyjnych.
rem	Root EM (Odniesienie do rdzenia)	Główna jednostka rekomendowana z racji dostępności systemu odczytu. Rozmiar jest odliczeniem iloczynu wariantu wielokrotności dla <code>font-size</code> na węźle jądrowym (tag html) w dokumencie głównym. Jeśli w specyfikacji klienta ustalono uwarunkowanie systemowe 20px, odwołanie zjawiskowe <code>2rem</code> zawsze na każdym slajdzie zapewni wyrzut z rzędu w locie 40px, ignorując pułapki strukturalne i ułatwiając zachowanie wymogów z Accessibility.

13.3. Palety Barw, Kanał Alpha i Zarządzanie Motywami

Kaskadowe Arkusze oferują cztery główne domeny wdrożeń wektorów barwnych wykorzystywanych na poziomie prezentacji odcieni w elementach interfejsu przeglądarki.

- 1. **Słownik Predefiniowany** – Zakres barw określonych słownie (anglojęzyczne). Słownik wykorzystywany u procesów prototypowych testów i walidacji elementów blokowych (np. weryfikowanie granicy w tle, `red`). Z racji wysoce podstawowych odcieni pomijany w gotowych do kompilacji produktach deweloperskich.
- 2. **HEX (System Szesnastkowy)** – Rozszerzona formuła odwołania za pośrednictwem znaku skrótowego `#`, używana natywnie we wspieraniu programów do prototypowania UX/UI. Czerń określono natywnie z użyciem kodu wyczerpania diody na matrycach RGB (`#000000`). Kolor w rzucie z maksimum rejestrów wygeneruje dla interfejsu powszechną biel we wszystkich warstwach dokumentacji dla węzłów (`#FFFFFF`).
- 3. **RGB / RGBA** – Otwiera zdolność parametrycznej strukturyzacji z dodaniem KANAŁU ALFA oznaczającym wsparcie warstw z półprzezroczystością powłoki kryjącej (np. kanał z 0.5 jako połowa uwarunkowania nieprzezroczystości z nałożonym zjawiskiem w ujęciu okien systemowych w modalu tła interfejsu na główny element warstwy wizualnej `rgba(0, 0, 0, 0.5)`).
- 4. **HSL** (Hue, Saturation, Lightness) – Operowanie współczynnikami z matematycznego wariantu natężenia odcienia, nasycenia z wysoce profesjonalnym zarządzaniem z wektora zmiany natężenia blasku i wagi nasilenia cieni z wariantów rzutowanych bezpośrednio na wyliczeniach z CSS.

Deklaracja Systemowych Zmiennych Natywnych (Custom Properties) Technologia wbudowana z rdzenia architektury, zezwala na tworzenie predefiniowanych kluczy zapisu dla wektorowych wdrożeń z wymuszonym odniesieniem w obszarach kaskadowych w procesie ewaluacji dla silnika.

```
/* W bloku głównym root odkładane zostają wskaźniki powielane jako pule odwołań kolorystycznych. */
:root {
  --primary-color: #007bff;
  --background-surface: #f8f9fa;
  --global-padding: 1.5rem;
}

/* Interfejs wymusza reaktywność na preferencje systemowe motywu nocnego w zjawiskach pobrania konfiguracji API. Zmiana nasycenia w systemie OS załączy natychmiast po zmianach. */
@media (prefers-color-scheme: dark) {
  :root {
```

```

    --primary-color: #66b0ff;
    --background-surface: #121212;
  }
}

.interface-button {
  background-color: var(--primary-color);
  padding: var(--global-padding);
}

```

13.4. Abstrakcyjne Funkcje Obliczeniowe

Implementacja logiki obliczeniowej przenosi część przeliczników na silnik w fazie renderowania z asynchronicznie przeliczanym czasem alokacji:

- `calc()` – Operacja matematyczna pozwalająca wariantom miksować odmienne typologie ze zmiennych np. wyliczając `width: calc(100vh - 50px)`.
- `clamp(MIN, IDEAL, MAX)` – Stosowany rygorystycznie do powszechnego zjawiska Fluid Typography (Wielkość płynna wymiarowania bez ucieczki z przedziałów po osiągnięciu granicy rzutu z odcięcia wymiaru dla sprzętów z okien powszechnych urządzeń przenośnych, zatrzymując się precyzyjnie bez wymuszeń w skali MediaQuery). Np. `font-size: clamp(1.5rem, 5vw, 3rem);`.

14. Typografia i Tekst w CSS

Narzędzia uwarunkowane w systemie do pełnej manipulacji rzutowaniem i kompozycją ciągów tekstowych bez interwencji znaczników przestarzałych.

14.1. Zewnętrzne Biblioteki Typograficzne (Google Fonts)

Standardowy ubiór oparty z racji powszechności na wbudowanych zestawach OS nie sprostą zjawiskom odznaczanym przez standard web-designu. Pliki dystrybuowane na zewnętrznych kanałach dostępu (CDN) zapewniają asynchroniczne załadowanie na styk od wdrożenia w drzewie asynchronicznego head.

```

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;600&display=swap');

body {
  /* W wypadku niedostępności zewnętrznego źródła w wariantcie usterki, zaimplementowano stos upadku (Font-Stack). Silnik pobierze
  z biblioteki systemowej natywne odpowiedniki. W przypadku czcionki sans-serif zastosuje literę bezszeryfową dla surowego obrysu bez
  wykończeń. */
  font-family: 'Inter', system-ui, -apple-system, sans-serif;
}

```

14.2. Sterowanie Prezentacją Wyrazową i Kształtami

ATRYBUT RZUTU DOKUMENTACJI	PARAMETRY W IMPLEMENTACJI RZUTOWANIA DLA ARCHITEKTA	LOGICZNA DEFINICJA DZIAŁANIA INTERFEJSÓW W SPECYFIKACJI
<code>font-weight:</code>	<code>400</code> (zwykły parametr sztywności normatywnej), <code>700</code> (zwiększone natężenie sztywności ołówkowej fontu - pogrubienie binarne).	Nasilenie strukturalne i natężenie gruboziarniste znaku zdefiniowanego na zjawisko wizualnego pogrubienia.
<code>font-style:</code>	<code>normal</code> , <code>italic</code>	Wariant użycia nachylenia, symulujący emfaticzne odniesienie tekstu.
<code>text-transform:</code>	<code>uppercase</code> , <code>lowercase</code> , <code>capitalize</code>	Transformacja logiki wyświetlenia od razu po procesie parsowania w zjawisku renderu z narzuceniem kapitalizacji z silnika bazy. Przeobraża małe znaki wyjściowe w bezwarunkowe pozycje zapisu u rzutowania interfejsów do wektora znakowego z wielkimi ciągami wieloliterowymi.
<code>text-align:</code>	<code>left</code> , <code>right</code> , <code>center</code> , <code>justify</code>	Przyporządkowanie do skrajni węzłów nadrzędnych. Ułożenie dokumentu we właściwych orientacjach krawędzi bloku, zapewniających dopasowanie rzutu na symetrię środkowe lub wyjustowanie tekstu ciągłego.
<code>line-height:</code>	<code>1.5</code> , <code>1.6</code> , <code>2</code>	Współczynnik interlinii na warstwach czytelniczych z bloków tekstu długiego. Dozwolone bez użycia sztywnego wymuszenia dziedziczenia we wskaźnikach z parametru rozmiaru np. 1.5 bez wymiarów ułamkowych z px, rzuca bezpieczne marginesowanie między wierszami po asymetrycznym przeliczeniu z <code>font-size</code> .
<code>letter-spacing:</code>	<code>1px</code> , <code>-0.5px</code>	Definicja operacyjna określająca rzutowane luki przestrzenne izolujące każdy pojedynczy wskaźnik alfabetu z sąsiadującymi braćmi na tym samym wektorze z wyrazu.
<code>text-decoration:</code>	<code>none</code> , <code>underline</code> , <code>line-through</code>	Graficzna dekoracja wpisania. Konfigurowalna pod kątem wyłączeń wektorów liniowych. Wykorzystywane głównie do wyłączenia narzucanych dolnych adnotacji od zjawisk referencji kotwicznych. Używane w notacjach przekreślenia cennych wartości u e-commerce'u z <code>line-through</code> .
<code>white-space:</code>	<code>nowrap</code> , <code>pre-wrap</code>	Kontrola spacji wierszowych w HTML. Funkcja chroniąca na wariantach predefiniowanych załamań na akapitach blokowych przed pęknięciem. Zablockowanie łamania u <code>nowrap</code> poskutkuje wymuszonym overflow elementu na siatce przy obniżaniu krawędzi bocznej systemu.

15. Model pudełkowy (Box Model)

Każdy element na stronie internetowej, niezależnie od tego, czy jest to nagłówek, paragraf, czy kontener typu `div`, przeglądarka traktuje jako prostokątne pudełko. Zrozumienie budowy tego modelu jest kluczowe dla poprawnego tworzenia układów (layoutów) za pomocą Flexbox lub Grid.

Strukturę tego modelu można łatwo przeanalizować, otwierając Narzędzia Deweloperskie przeglądarki (klawisz `F12`), a następnie przechodząc do zakładki `Elements`. Znajduje się tam graficzna wizualizacja modelu pudełkowego wybranego elementu.

15.1. Warstwowa budowa modelu pudełkowego

Model pudełkowy składa się z czterech głównych warstw, rozszerzających się od wewnątrz na zewnątrz:

1. Obszar zawartości (Content Box) Jest to wewnętrzna strefa elementu, w której fizycznie znajduje się tekst, grafika lub kolejne zagnieżdżone elementy HTML. Wymiary tej przestrzeni określone są najczęściej za pomocą właściwości `width` (szerokość) i `height` (wysokość). W domyślnym modelu CSS (`box-sizing: content-box`), zadeklarowana szerokość odnosi się wyłącznie do tej warstwy, ignorując dodatki na obrzeżach.

2. Dopełnienie wewnętrzne (Padding) Jest to pusta przestrzeń oddzielająca zawartość elementu od jego obramowania (`border`). Dopełnienie zwiększa sumaryczne wymiary pudełka, ale przejmuje kolor tła elementu (`background-color`). Ustawiając np. `padding: 10px 20px`, zapewniamy elementom odpowiedni odstęp wewnętrzny, co jest kluczowe dla estetyki tekstu wewnątrz przycisków czy paneli.

3. Obramowanie (Border) Obramowanie okala dopełnienie wewnętrzne i zawartość. Oznacza krawędź elementu, poza którą tło elementu już nie sięga. Szerokość obramowania (np. `border: 2px solid black`) dodaje się do całkowitego fizycznego rozmiaru

renderowanego prostokąta w drzewie DOM.

4. Margines zewnętrzny (Margin) Margines to całkowicie przezroczysta przestrzeń na zewnątrz elementu, która służy do odpychania sąsiadujących kontenerów od siebie (np. `margin-bottom: 15px;` oddziela wizualnie od siebie dwa paragrafy). W przeciwieństwie do paddingu, margines nigdy nie dziedziczy koloru tła – zawsze przyjmuje kolor przestrzeni za elementem.

15.2. Problem domyślnego modelu liczenia i rozwiązanie `box-sizing`

W domyślnym, historycznym podejściu (tzw. `content-box`), całkowita wielkość elementu to suma jego zadeklarowanej szerokości (`width`), dopełnienia (`padding`) oraz ramki (`border`). Prowadzi to do częstych problemów. Jeżeli kontener ma zadeklarowane `width: 50%` szerokości ekranu, a następnie dodamy do niego `padding: 10px` i `border: 2px`, jego fizyczny wymiar wyniesie 50% + 12px (dla każdej ze stron). Element rozszerzy się ponad zaplanowany wymiar, co spowoduje rozbieżność struktury siatki (np. zrzucenie sąsiedniego elementu do nowej linii).

Rozwiązanie: `box-sizing: border-box` Aby temu zapobiec, we współczesnym programowaniu frontendowym stosuje się powszechny reset właściwości `box-sizing`. Przestawia on mechanizm matematyczny silnika przeglądarki. Po jego użyciu zadeklarowana szerokość (np. `width: 500px`) staje się wymiarem absolutnym, nienaruszalnym i końcowym. Każde dodanie wewnętrznego dopełnienia czy ramki pomniejszy przestrzeń obszaru zawartości do wewnątrz, zamiast rozszerzać element na zewnątrz.

```
/* Wzorec powszechnie zalecany i używany na początku każdego projektu */
*, ::before, ::after {
  /* Utrzymuje wymiary z width/height jako wartości finalne włączając padding i border */
  box-sizing: border-box;

  /* Resetuje domyślne, niepożądane marginesy przeglądarek dla wszystkich elementów */
  margin: 0;
  padding: 0;
}
```

16. Layout i Architektura – CSS Flexbox oraz CSS Grid

Współczesny rozwój interfejsów webowych wykreował zapotrzebowanie na zaawansowane systemy tworzenia struktur siatkowych. Historyczne rozwiązania opierające się o właściwość `float` generowały znaczne trudności i komplikacje związane z pozycjonowaniem i responsywnością. W odpowiedzi na te problemy do standardu wprowadzono mechanizmy układu jednowymiarowego (Flexbox) oraz układu dwuwymiarowego (Grid).

16.1. Flexbox (Flexible Box Layout Module)

Standard zoptymalizowany pod kątem projektowania jednowymiarowych struktur przestrzennych (zarządzania układem wyłącznie w kontekście jednego wiersza lub pojedynczej kolumny w danym momencie). Elastycznie dostosowuje wielkości elementów podrzędnych do wymiarów kontenera nadrzędnego.

Definiowanie elementu nadrzędnego jako Flex Container uaktywnia ten model dla jego bezpośrednich potomków (Flex Items):

```
.flex-container {
  display: flex;
}
```

Kluczowe właściwości modułu Flexbox:

WŁAŚCIWOŚĆ NA KONTENERZE	LOGIKA DZIAŁANIA I PARAMETRY KONFIGURACYJNE	WARTOŚCI STANDARDOWE
<code>flex-direction</code>	Wyznacza główną oś układu, determinując wektor przepływu elementów bezpośrednich wewnątrz kontenera rodzica.	<code>row</code> (domyślnie, układa poziomo od lewej), <code>row-reverse</code> , <code>column</code> (ustawia elementy w układzie wertykalnym od góry, popularne na urządzeniach mobilnych), <code>column-reverse</code> .
<code>justify-content</code>	Odpowiada za dystrybucję wolnej przestrzeni i układ elementów wzdłuż wytyczonej Osi Głównej (Main Axis). Decyduje o symetrii wyrównywania bloków we wdrożeniach strukturalnych.	<code>flex-start</code> , <code>flex-end</code> , <code>center</code> , <code>space-between</code> (pierwszy i ostatni element przylegają bezpośrednio do krawędzi, a przestrzeń jest rozdysponowywana równo pomiędzy nimi), <code>space-around</code> .
<code>align-items</code>	Precyzuje wyrównanie elementów względem Osi Poprzecznej (Cross Axis), czyli wektora prostopadłego do osi wiodącej. Definiuje zachowanie bloków potomnych w obrębie wolnej linii wysokości.	<code>stretch</code> (rozciągnięcie do wysokości kontenera, opcja domyślna), <code>flex-start</code> (dociągnięcie w stronę początkowej krawędzi wertykalnej), <code>center</code> (absolutne wyśrodkowanie w pionie - wykorzystywane regularnie do pozycjonowania zawartości tekstowych na poziomie ikon), <code>flex-end</code> .
<code>gap</code>	Ustanawia sztywną krawędź oddzielającą i wymiaruje powtarzalne luki przestrzenne izolujące wszystkie sąsiadujące bloki w obrębie węzła. Eliminuje to wadliwe nawyki z narzucaniem klasycznych marginesów (margin) bocznych u elementów podrzędnych.	Np. <code>gap: 20px;</code> , <code>gap: 1.5rem;</code>
<code>flex-wrap</code>	Kontroluje załamanie ułożenia elementów podczas utraty elastyczności kontenera przy ograniczonym obszarze (wymusza zjawisko zawijania krawędzi do następnej powłoki linii podziału na Osi Krzyżowej).	<code>nowrap</code> (domyślnie dąży do kompresji elementów w jednej płaszczyźnie poziomej poza tolerancję rozmiaru), <code>wrap</code> (pozwala węzłom niekompatybilnym ulec zrzutowi strukturalnemu na drugi odcinek niżej).

Środek ciężkości interfejsów (Centrowanie Absolutne): Implementacja modelu redukuje wieloletni problem z wyrównaniem blokowym w pionie i w poziomie. W celu zrealizowania scentrowania idealnego we wdrożeniach wystarczy uaktywnić:

```
.absolute-center-wrapper {  
  display: flex;  
  justify-content: center; /* Wyśrodkowanie osi horyzontalnej */  
  align-items: center; /* Wyśrodkowanie osi wertykalnej */  
}
```

16.2. CSS Grid Layout Module

Zjawisko ewolucyjne nakierowane na dwuwymiarowe zarządzanie perspektywą, operujące jednoczesną dystrybucją wierszy (Rows) i kolumn (Columns). Wzorzec doskonały do makro-kompozycji, architektury stelaża interfejsów oraz stron wykorzystujących sidebar, header czy układy e-commerce.

Aktywacja systemu odbywa się przez deklarację kontenera siatki w rzucie z zagnieżdżenia:

```
.grid-container {  
  display: grid;  
}
```

Jednostka Proporcji Ułamkowej (fr – Fraction Unit) Siatka operuje elastyczną i nowatorską dyrektywą wielkości odniesienia `fr`, ułatwiając podział bloków i kompozycji bez precyzyjnych kalkulacji i obliczeń pikselowych. Przydzielenie np. `1fr 1fr 1fr` spowoduje geometryczne wyrównanie wymiarowania w trzech rzędach po równą wartość obszaru dostępnego w interfejsie po odliczeniu marginesów własnych luki (gap).

Szablony Stref Grid (Grid-Template-Areas) Dla skomplikowanych budów aplikacyjnych na bazie zjawisk obszarów, zastosowano semantyczne projektowanie siatki rzutów stref, umożliwiając deweloperom zdefiniowanie obszarów znakowych za pomocą stringów

mapujących nazwy poszczególnych komponentów. Pozwala to na uniknięcie operowania rzędami przez sztywne współrzędne liczb.

```
/* Struktura bazowa reprezentująca główny szkielet strony HTML <main> zawierającej górne menu (Header), sekcję filtrów bocznych (Sidebar), środek ze sprzedażą artykułów (Main Content) i Stopkę */
.app-container-html-grid {
  display: grid;

  /* Dystrybucja uwarunkowana z trójosiowej budowy siatki wertykalnej: Górny header utrzymuje 80px, środkowa powłoka dla zawartości sklepu automatycznie dystrybuuje wolną przestrzeń za pomocą 1fr (bądź auto), natomiast na krańcu dół otrzymuje pułap bezpieczny 50px */
  grid-template-rows: 80px 1fr 50px;

  /* Architektura dwóch sekcji: lewy wąski kontener na nawigację odciętą sztywno wartością 250px oraz prawa główna treść pozerająca elastyczną alokację reszty ujęcia horyzontalnego. */
  grid-template-columns: 250px 1fr;
  gap: 15px;

  /* Rozpisanie schematu stref operacyjnego odwołania do wytyczonych bloków wymiarowania wyżej */
  grid-template-areas:
    "header-area header-area"
    "sidebar-area main-content-area"
    "footer-area footer-area";
}

/* Kwalifikacja konkretnych elementów potomnych w DOM dla węzła obszaru o wytyczonych wyżej strefach */
.header-site { grid-area: header-area; background: red; }
.sidebar-filters { grid-area: sidebar-area; background: blue; }
.main-store { grid-area: main-content-area; background: yellow; }
.footer-site { grid-area: footer-area; }
```

Połączenie wdrożonych węzłów we wspólnym mianowniku pozwala np. obiektowi `header-site` na dynamiczne asymilowanie wymiarów na siatce kolumnowej obejmując dwie sekcje w procesie scalania (spanning). Obiekt docelowy podda się re-formacji pozycjonowania zachowując strukturę bezpieczną na powiązanych wdrożeniach strefowych.

17. Pozycjonowanie i System Współrzędnych (Position)

Zastosowanie koncepcji Flexbox i Grid nie zaspokaja całości wyzwań związanych z architekturą interfejsów, szczególnie w kontekście elementów, które wymagają całkowitego oderwania od standardowego porządku układu dokumentu (np. pływające nagłówki, elementy modalne, ikony notyfikacji powiązane bezpośrednio ze wskaźnikami punktowymi rodzica). Realizacja tych układów opiera się o własność `position`. Zmiana tej dyrektywy wyłącza standardowe zachowanie (Flow) rzutowane na dany obiekt.

- `position: static;` – Domyślny tryb renderowania dla każdego węzła tagu HTML. Element respektuje naturalny przepływ dokumentu uwarunkowany przez siatki rodzica i zasady rzutowania. Atrybuty uwarunkowane zjawiskami osiowymi (np. `top`, `bottom`, `left`, `right`) a także współczynnik osi Z (`z-index`) nie generują absolutnie żadnego wpływu na położenie.
- `position: relative;` – Przesunięcie wektorowe odniesione bezwzględnie w stosunku do pierwotnego, oryginalnego położenia elementu. Użycie atrybutu `top: 10px;` skutkuje wizualnym wymuszeniem wyjścia w górę osi wertykalnej bez zakłócenia układu i pozostawienia naturalnej wyrwy, jaką obiekt ten zajął w hierarchii braterskiej (sąsiednie elementy zachowują się, jakby dany węzeł wciąż pozostawał w statycznym wymiarowaniu spoczynkowym pierwotnej lokalizacji). **Kluczowa cecha operacyjna:** Obiekt tak zdefiniowany pełni fundamentalną rolę układu odniesienia i przestrzeni ograniczającej dla elementów potomnych zdefiniowanych w układzie `absolute`.
- `position: absolute;` – Bezwzględne zerwanie i wymuszenie wyjęcia elementu graficznego poza schemat standardowego przepływu, co doprowadza do dezaktywacji uwarunkowań grawitacyjnych dla wykluczonego podmiotu DOM. Obszar, który obiekt by w naturalnym środowisku wypełniał, zostaje zasymilowany przez otoczenie strukturalne (pozbawiony jest fizycznego wskaźnika powiązań obrysu rozmiaru na systemach nadrzędnych). Pozycjonowanie rzutu odwotawczego dla osi X i Y dokonywane jest nie na

krawędzi głównego okna systemu, lecz w kontekście pierwszego napotkanego na drzewie zagnieżdżeń rodzica z określonym systemem odniesienia (`position` z właściwością nierówną domyślnej `static` - np. `relative`). O ile w relacji przodków takiego odniesienia deweloper nie nakreślił - pozycjonowanie na krawędzi dokumentu powraca jako wyjściowa rama domyślna.

- `position: fixed;` – Wykluczenie strukturalne całkowite oparte o parametryczne umiejscowienie graficzne trwale wyeksponowane względem obrysu przestrzeni roboczej przeglądarki użytkownika. Wyklucza się na nim wszelkiej formy przeliczniki zachowań w drzewie. Niezależnie od zjawiska uwarunkowania systemu pasków przewijania (Scrollbars) w długim odczycie uwarunkowanego wymiarami pionowymi czy poziomymi element operacyjny trwale przytwierdzony będzie wektorem współrzędnych interfejsu (często powszechnie stosowane pod kątem przycisków ewakuacyjnych, podłogowych z menu aplikacji czy paska nawigacji nadrzędnej osadzonego na zerowej pozycji z `top: 0`).
- `position: sticky;` – Konfiguracja układu hybrydowego w odniesieniu zachowań przewijanych powłok elementu rzutu względem zdefiniowanych wymuszeń systemowych tolerancji. Występuje jako klasyczny rzut grawitacyjny w koncepcji zachowań natywnych (`static`) aż w momencie wystąpienia interakcji skrolowania w którym obrys przekroczy próg definicyjnej offsetowej (np. `top: 0`), obiekt dokonuje przeobrażenia i przejmuje cechy przypisywane parametrom blokad powłok z wariantu `fixed`. Jest to zjawisko wykorzystywane przede wszystkim we wdrożeniach architektonicznych stref podążających nagłówków i wskaźników nawigacji odrzucających powrót na wierzchołek sekcji.

18. Responsywność i Architektura Uwarunkowana (Responsive Web Design)

Paradygmat Responsive Web Design (RWD) narzuca obligatoryjne wdrożenia i normy architektoniczne umożliwiające prezentację jednolitego wektora źródłowego HTML dla zróżnicowanych przedziałów ekranowych i zjawisk rzutowych od niewielkich rozdzielczości z platform przenośnych do obszernych powłok z natywnych wyświetlaczy środowisk biurkowych (Desktop). Eliminacja zjawiska utrzymywania izolowanych plików kodów aplikacyjnych (tzw. "mobilnych subdomen") to bezpośredni wynik użycia w pełni dynamicznej interpretacji i adaptacji za pośrednictwem kaskadowych dyrektyw mediów uwarunkowanych.

18.1. Implementacja wariantu konwersji perspektywy (`viewport`)

Ignorowanie implementacji specyficznego znacznika metadanych powoduje nałożenie silnikowych filtrów powiększalnika w domyślnych zachowaniach (szczególnie restrykcyjnego u środowisk iOS), symulujących wyświetlenie sztywnej rozdzielczości z powszechnej perspektywy monitora na poziomie obszaru ograniczonych przestrzeni systemów smartfonów, co niweczy całkowicie strukturę adaptacyjną. Parametr określa jednoznacznie ugodę rozdzielczą. Znacznik ten zawsze osadza się w warstwie sekcji bazowej `<head>`.

```
←!— Konfiguracja zjawiska perspektywy rzutu dla silników rzutowania wymusza natywne uwarunkowanie skali okna ramy i likwidację  
odgórnego systemu oszustw rozdzielczych skalowania przez ujęcie do wskaźnika jednostkowego →  
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

18.2. Punkty Przełamania i Dyrektywa Media Queries (`@media`)

Iniekcja kaskadowych filtrów wymuszających nadpisanie stylizacji odbywa się dzięki dyrektywie `@media`. Koncepcja zakłada powszechnie zalecaną w profesjonalnym tworzeniu struktury ideologię **Mobile-First**. W tej technice architektura początkowego ujęcia i kod globalnych atrybutów bez restrykcji `media` jest natywnie traktowany jako dedykowany dla ograniczeń i małych zrzutów przestrzennych. Dopiero kolejne wytyczne kaskadowo wstrzykują modyfikatory do rozleglejszych i większych okien na wytyczonych limitach przerwań.

```
/* ---- 1. KOMPOZYCJA STARTOWA (MOBILE-FIRST - ZAŁOŻENIA ARCHITEKTURY DLA OGRANICZONEJ POWIERZCHNI EKRANÓW MOBILNYCH) ---- */  
.products-database-wrapper {  
  /* Konfiguracja ukierunkowana natywnie na optymalne czytanie uwarunkowane ruchem jednowektorowym w osi wertykalnej dla  
  wyświetlaczy z ekranami o ograniczonych przestrzeniach dla uwarunkowania. Implementuje układ oparty na pojedynczej sekwencyjnej  
  ułożonej w kolumny bez pęknięć. */  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 10px;  
}  
  
.commercial-base-text {
```

```

    font-size: 1.2rem;
}

/* ---- 2. DEKLARACJA PUNKTU ZMIANY DLA ŚRODOWISKA ROZSZERZONEGO (TABLETY, EKRANY ŚREDNIE) ---- */
/* Algorytm wywoła i wstrzyknie kaskadę odczytów dla bloku stylów wyłącznie w wymuszonym wystąpieniu zjawiska poszerzenia ramy
okiennej ponad lub równej progu wyznaczonemu na 768 pikseli w parametrach systemów renderujących okno. */
@media (min-width: 768px) {
    .products-database-wrapper {
        /* Przechwycenie dyrektywy wejścia i nadpisanie konfiguracji układu na podział ułamkowy dystrybucji na podział dwóch
strefowych ram w kolumnach u szerokiej architektury u wektorów bocznych */
        grid-template-columns: 1fr 1fr;
    }
}

/* ---- 3. WARIANT DLA EKRANÓW STRUKTURY ŚRODOWISKA DESKTOP I MONITORÓW WYSOKIEJ ROZDZIELCZOŚCI ---- */
@media (min-width: 1200px) {
    .products-database-wrapper {
        /* Wymuszenie dyslokacji struktur bazy dla wykorzystania ogromnej luki wyświetlanego elementu z siatki na szerokie rzuty do
wyświetlania w kolumnie asortymentowej w wielokrotnej wariacji */
        grid-template-columns: repeat(4, 1fr);
    }
    .commercial-base-text {
        font-size: 3rem; /* Zwiększenie natężenia wymuszenia w architekturze literowej dostosowującej perspektywę z uwarunkowania
wyższego dystansu czytelnika z rzutu od wariantu z parametrów komputera osadzonego */
    }
}
}

```

18.3. Modułowe Wykluczenia Widoczności pomiędzy Architekturą (Drawer Menu)

Technika wymuszająca zjawisko rozdzielenia kompozycji i zarządzania ukrywaniem struktur widokowych odnosi się m.in. dla nawigacji mobilnej po menu nawigacyjnym "Hamburger Nav" operującym po logice wyłączenia. W środowiskach biurkowych (`min-width: 1200px`) zalecane jest powielenie standardowego drzewa z łączami sekcji (Lista odnośników wewnątrz Header). Elementy wyłączzonego systemu ukrywa się powszechnie implementując właściwość `display: none` dla wariantu wymuszonego rzutowania na wejścia mobilne. Podczas oceny kompozycji małego wyświetlacza z zastosowaniem ukrycia odrzuconej nawigacji asymiluje wyłączenie listy, włączając na to wejścia aktywne wariantu paska operacyjnego. Wynikiem działań warunkowych jest optymalizacja zarządzania architekturą układu wizualnego odcinająca asymetryczne deformacje dla niekompatybilnej rozdzielczości ekranowej użytkownika.

19. Przejścia CSS (Transitions) w Praktyce

Przejścia (Transitions) pozwalają na płynną zmianę wartości danej właściwości CSS w określonym czasie. Zamiast natychmiastowego przeskoku koloru lub wielkości, przeglądarka sama oblicza i rysuje klatki pośrednie.

19.1. Składnia właściwości `transition`

Najczęściej korzystamy ze skróconego zapisu `transition`, który łączy w sobie cztery elementy:

- `property` (co zmieniamy, np. `background-color`, `transform`, lub `all`),
- `duration` (czas trwania, np. `0.3s`),
- `timing-function` (krzywa tempa, np. `ease`, `linear`, `ease-in-out`),
- `delay` (opóźnienie startu, np. `0.1s`).

Przykład: Podstawowy przycisk z płynnym hoverem

```

.btn-primary {
    background-color: #007bff;
}

```

```

    color: white;
    padding: 10px 20px;
    border-radius: 5px;
    border: none;
    cursor: pointer;

    /* Płynnie zmieniamy tło i cień w czasie 0.3 sekundy */
    transition: background-color 0.3s ease, box-shadow 0.3s ease;
}

.btn-primary:hover {
    background-color: #0056b3;
    box-shadow: 0 4px 8px rgba(0,0,0,0.2);
}

```

19.2. Częste błędy - używanie `all`

Wielu początkujących programistów używa `transition: all 0.3s ease;`. Jest to błąd optymalizacyjny. Jeśli animujemy tylko kolor tła, zmuszanie przeglądarki do śledzenia wszystkich możliwych właściwości (nawet tych, które się nie zmieniają) pożera niepotrzebnie zasoby komputera. Zawsze precyzuj, co dokładnie animujesz.

19.3. React: Przejścia a warunkowe renderowanie klas

W Reactcie bardzo często podpinamy klasy CSS zależnie od stanu (State). Przejścia zadziałają idealnie, pod warunkiem, że element znajduje się w DOM.

```

import React, { useState } from 'react';
import './Card.css';

export default function InteractiveCard() {
    const [isActive, setIsActive] = useState(false);

    return (
        <div
            // Dodajemy klasę 'card--active' w zależności od stanu
            className={`card ${isActive ? 'card--active' : ''}`}
            onClick={() => setIsActive(!isActive)}
        >
            Kliknij mnie!
        </div>
    );
}

```

```

/* Card.css */
.card {
    width: 200px;
    height: 100px;
    background-color: #eee;
    transform: scale(1);

    /* Zdefiniowanie przejścia w klasie bazowej */
    transition: transform 0.2s ease-in-out, background-color 0.2s ease;
}

```

```
.card--active {
  background-color: #4CAF50;
  color: white;
  transform: scale(1.1); /* Karta lekko urosła */
}
```

20. Animacje i Keyframes

Podczas gdy `transition` działa tylko "od punktu A do punktu B" (np. z normalnego stanu do hovera), dyrektywa `@keyframes` pozwala stworzyć wieloetapowe animacje żyjące własnym życiem, zapętlone, lub uruchamiające się automatycznie przy załadowaniu strony.

20.1. Definiowanie klatek kluczowych (@keyframes)

Tworzymy siatkę czasu od `0%` do `100%`. W tych punktach określamy, jak ma wyglądać element.

```
/* Animacja trzęsącego się dzwonka powiadomień */
@keyframes ringBell {
  0% { transform: rotate(0deg); }
  15% { transform: rotate(15deg); }
  30% { transform: rotate(-15deg); }
  45% { transform: rotate(10deg); }
  60% { transform: rotate(-10deg); }
  75% { transform: rotate(5deg); }
  100% { transform: rotate(0deg); }
}
```

20.2. Wywoływanie animacji w elemencie

Samo napisanie `@keyframes` nic nie robi. Należy je podpiąć do konkretnej klasy za pomocą właściwości `animation`.

```
.notification-icon {
  display: inline-block;
  /* Uruchamiamy animację ringBell, ma trwać 1 sekundę, tempo ease-in-out, i powtarzać się w nieskończoność */
  animation: ringBell 1s ease-in-out infinite;
}
```

20.3. Pełna lista właściwości animacji

Skrót `animation` kryje w sobie bardzo wiele ustawień. Oto one rozbite na czynniki pierwsze:

- `animation-name`: nazwa naszego `@keyframes`.
- `animation-duration`: czas (np. `2s`).
- `animation-timing-function`: krzywa czasu (np. `linear`).
- `animation-delay`: opóźnienie przed startem (np. `0.5s`).
- `animation-iteration-count`: ile razy ma się wykonać (`infinite` oznacza w nieskończoność).
- `animation-direction`: kierunek odtwarzania (`normal`, `reverse`, `alternate` - czyli tam i z powrotem).
- `animation-fill-mode`: co się dzieje po zakończeniu animacji (najważniejsza to `forwards` - element zatrzymuje się na stylu z klatki 100% i tak już zostaje).

Przykład: Element pojawiający się od dołu (Fade In Up)

Bardzo popularny efekt przy wczytywaniu stron. Zastosujemy go w komponencie React.

```
/* Definicja animacji wpadnięcia */
@keyframes fadeInUp {
  0% {
    opacity: 0;
    transform: translateY(40px); /* Start 40px niżej */
  }
  100% {
    opacity: 1;
    transform: translateY(0); /* Koniec w oryginalnym miejscu */
  }
}

.fade-in-element {
  /* Animacja jednorazowa. Zatrzymuje się na 100% (forwards). */
  animation: fadeInUp 0.8s ease-out forwards;
}

/* Trik: Różne opóźnienia dla listy elementów */
.delay-1 { animation-delay: 0.1s; }
.delay-2 { animation-delay: 0.2s; }
.delay-3 { animation-delay: 0.3s; }
```

```
// React JSX
export default function FeaturesList() {
  return (
    <ul>
      /* Elementy będą "wpadać" jeden po drugim dzięki klasom delay */
      <li className="fade-in-element delay-1">Szybkość</li>
      <li className="fade-in-element delay-2">Bezpieczeństwo</li>
      <li className="fade-in-element delay-3">Skalowalność</li>
    </ul>
  );
}
```

21. Transformacje 2D i 3D

Właściwość `transform` służy do przesuwania, obracania, skalowania i pochylania elementów. Największą zaletą transformacji jest to, że nie wpływają one na układ innych elementów na stronie. Jeśli przesuniesz element za pomocą `transform`, reszta strony nawet tego nie "zauważy", co czyni te operacje niezwykle wydajnymi (korzystają z akceleracji karty graficznej - GPU).

21.1. Podstawowe funkcje 2D

```
.box {
  width: 100px;
  height: 100px;
  background-color: coral;
  transition: transform 0.3s ease;
}
```

```

/* 1. Przesunięcie (Translate) */
/* Przesuwa o 50px w prawo (oś X) i 20px w dół (oś Y) */
.box--translate:hover {
    transform: translate(50px, 20px);
}

/* 2. Obrót (Rotate) */
/* Obraca element o 45 stopni zgodnie z ruchem wskazówek zegara */
.box--rotate:hover {
    transform: rotate(45deg);
}

/* 3. Skalowanie (Scale) */
/* Powiększa element dwukrotnie (2.0) */
.box--scale:hover {
    transform: scale(2);
}

/* 4. Łączenie transformacji */
/* Ważne: Kolejność ma znaczenie! Najpierw przesunie, potem obróci. */
.box--combined:hover {
    transform: translateX(50px) rotate(90deg) scale(1.5);
}

```

21.2. Transformacje 3D (Perspektywa)

Aby uzyskać efekt głębi (przestrzeni 3D), musimy dodać właściwość `perspective` do rodzica elementu, który będziemy obracać. Perspektywa określa, jak daleko wirtualna kamera znajduje się od obiektu.

Przykład: Odwracająca się karta (Flip Card)

To klasyczny przykład interfejsu (np. karta produktu, która po najechaniu odwraca się, pokazując specyfikację z tyłu).

```

// Karta w React
export default function FlipCard() {
    return (
        <div className="flip-card-container">
            <div className="flip-card-inner">
                {/* Przód karty */}
                <div className="flip-card-front">
                    
                    <h3>Marek</h3>
                </div>
                {/* Tył karty */}
                <div className="flip-card-back">
                    <p>Programista Front-End</p>
                    <p>React, CSS, Node.js</p>
                </div>
            </div>
        </div>
    );
}

```

```

/* Kontener - nadaje perspektywę całemu widokowi */
.flip-card-container {
  width: 300px;
  height: 400px;
  perspective: 1000px; /* Im mniejsza wartość, tym silniejszy efekt 3D (rybie oko) */
}

/* Wewnętrzny div, który faktycznie się obraca */
.flip-card-inner {
  width: 100%;
  height: 100%;
  transition: transform 0.8s;
  transform-style: preserve-3d; /* Ważne: pozwala dzieciom zachować trójwymiarowość */
  position: relative;
}

/* Po najechaniu na główny kontener, obróć środek o 180 stopni wzdłuż osi Y */
.flip-card-container:hover .flip-card-inner {
  transform: rotateY(180deg);
}

/* Wspólne style dla przodu i tyłu */
.flip-card-front, .flip-card-back {
  width: 100%;
  height: 100%;
  position: absolute;
  /* Ukrywa tylną stronę elementu, gdy jest on odwrócony plecami do użytkownika */
  backface-visibility: hidden;
  border-radius: 10px;
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}

/* Przód - normalny */
.flip-card-front {
  background-color: white;
}

/* Tył - jest z definicji obrócony o 180 stopni, żeby tekst nie był lustrzanym odbiciem! */
.flip-card-back {
  background-color: #2980b9;
  color: white;
  transform: rotateY(180deg);
}

```

22. Tła, Gradienty i Filtry Wizualne

CSS potrafi generować złożoną grafikę bezpośrednio w przeglądarce, eliminując potrzebę ładowania ciężkich obrazków (np. w formacie PNG) tylko po to, by uzyskać rozmyte tło czy nałożony kolor.

22.1. Gradienty w CSS

Gradienty traktowane są w CSS jako obraz (`background-image`), a nie zwykły kolor (`background-color`).

```

/* 1. Gradient Liniowy (Linear Gradient) */
/* Przechodzi od lewego górnego rogu (135 stopni) w dół */
.bg-linear {
  background-image: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

/* 2. Gradient Promienisty (Radial Gradient) */
/* Rozchodzi się ze środka w kształcie koła */
.bg-radial {
  background-image: radial-gradient(circle at center, #ff0844 0%, #ffb199 100%);
}

/* 3. Gradient Stożkowy (Conic Gradient) - idealny do wykresów kołowych */
.pie-chart {
  width: 150px;
  height: 150px;
  border-radius: 50%; /* Robimy idealne koło */
  background-image: conic-gradient(
    #4CAF50 0deg 90deg, /* Pierwsze 25% (od 0 do 90 stopni) zielone */
    #2196F3 90deg 270deg, /* Kolejne 50% niebieskie */
    #FFC107 270deg 360deg /* Ostatnie 25% żółte */
  );
}

```

22.2. Wiele tła naraz (Multiple Backgrounds)

Do jednego elementu możesz przypisać kilka tła oddzielonych przecinkiem. Rysowane są one na warstwach – pierwsze tło na liście jest "najwyżej" (najbliżej użytkownika).

```

.hero-section {
  height: 100vh;
  background:
    /* Górna warstwa: Półprzezroczysty, ciemny gradient nakładający się na obrazek */
    linear-gradient(rgba(0,0,0,0.6), rgba(0,0,0,0.6)),
    /* Dolna warstwa: Faktyczne zdjęcie tła */
    url('/images/office.jpg') center/cover no-repeat;

  color: white;
  display: flex;
  justify-content: center;
  align-items: center;
}

```

22.3. Filtry CSS (`filter`) i Backdrop Filter

Filtry pozwalają na modyfikację obrazka w czasie rzeczywistym, podobnie jak w Photoshopie.

```

.image-gallery img {
  width: 300px;
  height: 200px;
  /* Obrazek domyślnie jest czarno-biały */
}

```



```

    filter: grayscale(100%);
    transition: filter 0.3s ease;
}

.image-gallery img:hover {
    /* Po najechaniu odzyskujemy kolory i lekko podkręcamy jasność */
    filter: grayscale(0%) brightness(110%);
}

/* Cień wektorowy (Drop Shadow) */
/* Różnica względem box-shadow polega na tym, że drop-shadow otacza niepuste piksele obrazków PNG (np. wycięte logo), a box-shadow rysuje zawsze prostokąt. */
.logo-png {
    filter: drop-shadow(0 10px 15px rgba(0,0,0,0.5));
}

```

Rozmycie Tła - Glassmorphism (`backdrop-filter`)

Bardzo popularny efekt w nowoczesnym designie. Tworzy wrażenie zmatowionej szklanej tafli, przez którą widać to, co znajduje się pod spodem.

```

.glass-panel {
    /* Częściowo przezroczyste białe tło */
    background-color: rgba(255, 255, 255, 0.1);

    /* Rozmycie wszystkiego, co znajduje się FIZYCZNIE POD panelem */
    backdrop-filter: blur(10px);
    -webkit-backdrop-filter: blur(10px); /* Wsparcie dla Safari */

    border: 1px solid rgba(255, 255, 255, 0.2);
    border-radius: 15px;
    padding: 2rem;
    box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.37);
}

```

23. Zmienne CSS i Dynamiczny Motyw (Dark Mode)

Zmienne CSS (zwane profesjonalnie Niestandardowymi Właściwościami - *Custom Properties*) pozwalają na zdefiniowanie wartości w jednym miejscu i używanie jej wielokrotnie w całym dokumencie. Różnią się one od zmiennych w SASS tym, że działają w czasie rzeczywistym i można je łatwo nadpisywać za pomocą JavaScriptu lub pseudoklas (np. przy przełączaniu motywu).

23.1. Definiowanie i używanie zmiennych

Zmienne zazwyczaj definiuje się w głównym elemencie dokumentu, używając pseudoklas `:root` (co odpowiada tagowi `<html>`). Nazwa zmiennej zawsze musi zaczynać się od dwóch myślników `--`.

```

:root {
    --primary-color: #007bff;
    --text-main: #333333;
    --bg-main: #ffffff;
    --spacing-md: 16px;
    --border-radius: 8px;
}

```

```

.button {
  /* Używamy funkcji var(), aby pobrać wartość zmiennej */
  background-color: var(--primary-color);
  padding: var(--spacing-md);
  border-radius: var(--border-radius);
  color: white;
}

.article-text {
  color: var(--text-main);
}

```

23.2. Praktyczny Dark Mode (Tryb Ciemny) w React

Oto bardzo prosty i wydajny wzorec, jak stworzyć przełącznik trybu ciemnego, wykorzystując klasę dopinaną do najwyższego kontenera (lub samego elementu `body`) oraz zmienne CSS.

```

/* Domyślny, jasny motyw */
:root {
  --bg-color: #ffffff;
  --text-color: #121212;
  --card-bg: #f5f5f5;
}

/* Kiedy element body otrzyma klasę 'dark-theme', zmienne ZOSTANĄ NADPISANE */
body.dark-theme {
  --bg-color: #121212;
  --text-color: #ffffff;
  --card-bg: #1e1e1e;
}

/* Style komponentów korzystają wyłącznie ze zmiennych! Nic nie wiedzą o klasie 'dark-theme' */
.app-container {
  background-color: var(--bg-color);
  color: var(--text-color);
  min-height: 100vh;
  transition: background-color 0.3s ease, color 0.3s ease;
}

.card {
  background-color: var(--card-bg);
  padding: 20px;
  border-radius: 10px;
}

```

```

import React, { useState, useEffect } from 'react';

export default function App() {
  const [isDark, setIsDark] = useState(false);

```

```
// Kiedy zmienia się isDark, dodajemy/usuwamy klasę z tagu <body>
useEffect(() => {
  if (isDark) {
    document.body.classList.add('dark-theme');
  } else {
    document.body.classList.remove('dark-theme');
  }
}, [isDark]);

return (
  <div className="app-container">
    <h1>Witaj w mojej aplikacji</h1>
    <div className="card">
      <p>To jest przykładowa karta.</p>
    </div>

    <button onClick={() => setIsDark(!isDark)}>
      Zmień motyw na {isDark ? 'Jasny' : 'Ciemny'}
    </button>
  </div>
);
}
```

24. Wzorce Układów: Flexbox Masterclass

Znajomość właściwości to jedno, a umiejętność ich praktycznego zastosowania to drugie. Poniżej znajdują się gotowe przepisy na najpopularniejsze układy stron i komponentów przy użyciu modułu Flexbox.

24.1. Idealne wyśrodkowanie (Perfect Center)

Ten problem trapił programistów przez dekady. Za pomocą trzech linii we Flexboxie centrujemy element zarówno w poziomie, jak i w pionie.

```
.center-wrapper {
  display: flex;
  justify-content: center; /* Wyśrodkowanie na osi X */
  align-items: center;     /* Wyśrodkowanie na osi Y */

  height: 100vh;          /* Element musi mieć wysokość, by móc centrować w pionie! */
  background-color: #eee;
}

.centered-box {
  padding: 30px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
```

24.2. Rozkład "Space Between" (Pasek nawigacji)

Kiedy masz logotyp po lewej stronie, a linki nawigacyjne po prawej stronie, używasz właściwości rozkładania wolnej przestrzeni.

```
export function Navbar() {
  return (
    <nav className="navbar">
      <div className="navbar-logo">MojaFirma</div>
      <ul className="navbar-links">
        <li><a href="#">0 nas</a></li>
        <li><a href="#">Usługi</a></li>
        <li><a href="#">Kontakt</a></li>
      </ul>
    </nav>
  );
}
```

```
.navbar {
  display: flex;
  justify-content: space-between; /* Wypycha skrajne elementy do krawędzi */
  align-items: center;           /* Wyrównuje je w pionie */

  padding: 15px 30px;
  background-color: #333;
  color: white;
}

.navbar-links {
  display: flex;
  gap: 20px; /* Robi równy, 20-pikselowy odstęp pomiędzy linkami (genialna opcja!) */
  list-style: none;
  margin: 0;
  padding: 0;
}
```

24.3. "Sticky Footer" (Stopka przyklejona do dołu)

Gdy na stronie jest bardzo mało tekstu, stopka potrafi "podskańczyć" do połowy ekranu, zostawiając brzydką pustą przestrzeń poniżej. Wykorzystujemy `flex-grow`, by zmusić główną część strony do pochłonięcia całego pustego miejsca.

```
export function PageLayout() {
  return (
    <div className="site-wrapper">
      <header className="header">Witaj na stronie</header>

      <main className="main-content">
        To jedyne zdanie na tej stronie.
      </main>

      <footer className="footer">Stopka</footer>
    </div>
  );
}
```

```

.site-wrapper {
  display: flex;
  flex-direction: column; /* Ustawiamy układ w pionie (jeden pod drugim) */
  min-height: 100vh;      /* Minimalna wysokość to 100% ekranu przeglądarki */
}

.header {
  background-color: #2c3e50;
  color: white;
  padding: 20px;
}

.main-content {
  /* Magia! Mówimy głównej sekcji: "Urośnij i zabierz całe wolne miejsce!" */
  /* To automatycznie zepchnie stopkę na sam dół ekranu. */
  flex-grow: 1;

  padding: 20px;
}

.footer {
  background-color: #34495e;
  color: white;
  padding: 20px;
  text-align: center;
}

```

25. Wzorce Układów: CSS Grid Recipes

CSS Grid to absolutny fundament, jeśli chcemy układać elementy nie w jednej linii, lecz w szachownicy (rzędy i kolumny jednocześnie).

25.1. Płynna siatka kart (Auto-fit Grid) bez Media Queries!

To najważniejszy wzorec CSS Grid, używany w każdym sklepie internetowym czy galerii zdjęć. Rozkłada elementy automatycznie i łamie je do nowej linii na telefonach komórkowych bez używania ani jednej klauzuli `@media`.

```

.auto-grid {
  display: grid;
  gap: 20px;

  /*
   Tłumaczenie tej magicznej formuły:
   repeat(auto-fit, ...) - Przeglądarko, wciśnij w rząd tyle kolumn, ile tylko zdołasz.
   minmax(250px, 1fr) - Każda kolumna musi mieć MINIMUM 250px szerokości,
                        ale jeśli zostanie wolne miejsce, rozciągnij je po równo (1fr).
  */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
}

.grid-item {
  background-color: white;
  padding: 20px;
}

```

```
border-radius: 8px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
```

25.2. Układ dwukolumnowy "Sidebar + Content"

Gdy chcemy stworzyć klasyczną stronę z paskiem bocznym (sidebar) np. o szerokości 300px, a reszta strony ma być poświęcona na artykuł:

```
.page-layout {
  display: grid;
  /* Dwie kolumny: pierwsza na twardo 300px, druga zajmuje resztę (1fr) */
  grid-template-columns: 300px 1fr;
  gap: 30px;
}

/* W przypadku telefonów zmieniamy układ na kolumnowy (jeden pod drugim) */
@media (max-width: 768px) {
  .page-layout {
    /* Nadpisujemy: teraz jest to po prostu jedna kolumna na pełną szerokość */
    grid-template-columns: 1fr;
  }
}
```

26. Style w React: Moduły i CSS-in-JS

W tradycyjnym HTML dołączaliśmy po prostu jeden plik `style.css`. W dużych aplikacjach React, gdzie mamy setki komponentów, takie podejście powoduje konflikty nazw klas (tzw. wyciekanie stylów). Aby temu zapobiec, społeczność stworzyła trzy popularne standardy organizowania CSS.

26.1. Klasyczne klasy i BEM

Najprostsze podejście to wciąż zwykły CSS, ale pisany w metodyce BEM (Block, Element, Modifier), aby uniknąć konfliktów.

```
// Button.jsx
import './Button.css';

export function Button({ isPrimary, children }) {
  return (
    <button className={`btn ${isPrimary ? 'btn--primary' : 'btn--secondary'}`}>
      {children}
    </button>
  );
}
```

```
/* Button.css */
.btn {
  padding: 10px 20px;
  border-radius: 4px;
}

.btn--primary {
```

```

    background-color: blue;
    color: white;
  }
  .btn--secondary {
    background-color: gray;
    color: black;
  }
}

```

26.2. CSS Modules (Polecane dla początkujących)

To rozwiązanie wbudowane w narzędzia takie jak Vite czy Next.js. Tworzysz plik z rozszerzeniem `.module.css`. React podczas budowania aplikacji zmieni nazwy Twoich klas na unikalne (np. z `.card` zrobi `.Button_card__3fKz`). Nigdy więcej konfliktów!

```

/* UserProfile.module.css */
.card {
  background: white;
  border: 1px solid #ccc;
  padding: 20px;
}

.title {
  color: red;
}

```

```

// UserProfile.jsx
// Importujemy obiekt 'styles', w którym ukryte są nasze wygenerowane klasy
import React from 'react';
import styles from './UserProfile.module.css';

export function UserProfile() {
  return (
    <div className={styles.card}>
      <h2 className={styles.title}>Jan Kowalski</h2>
    </div>
  );
}

```

26.3. Podejście Utility-First: Tailwind CSS

Tailwind odwraca zasady gry. Zamiast pisać pliki CSS, używamy tysięcy gotowych, mikroskopijnych klas prosto w atrybucie `className`. Każda klasa odpowiada za jedną małą rzecz (np. `bg-blue-500` robi niebieskie tło, `p-4` dodaje padding, `rounded` robi zaokrąglone rogi).

To podejście jest uwielbiane przez programistów Reacta, bo nie trzeba w ogóle opuszczać pliku `.jsx`.

```

// Zwykły komponent w Tailwind CSS
export function TailwindCard() {
  return (
    <div className="max-w-sm rounded overflow-hidden shadow-lg bg-white p-6">
      <h2 className="font-bold text-xl mb-2 text-gray-800">
        Karta w Tailwindzie
      </h2>
    </div>
  );
}

```

```

    </h2>
    <p className="text-gray-700 text-base">
        Zbudowanie tej karty nie wymagało napisania ani jednej linijki w pliku .css!
    </p>
    <button className="mt-4 bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
        Czytaj więcej
    </button>
</div>

);
}

```

27. Walidacja Formularzy i API Przeglądarki

Nowoczesny HTML5 potrafi sam sprawdzać, czy formularz został poprawnie wypełniony, oszczędzając nam pisanie setek linijek w JavaScript. To tzw. *Constraint Validation API*.

27.1. Atrybuty walidacji w HTML

Wystarczy dodać odpowiednie słowa kluczowe do tagu `<input>`:

```

<form>
    <!-- Pole wymagane (nie można wysłać formularza, gdy jest puste) -->
    <input type="text" required placeholder="Twoje imię">

    <!-- Długość znaków (min. 8, max. 20) -->
    <input type="password" minlength="8" maxlength="20" required>

    <!-- Ograniczenia liczbowe (tylko od 1 do 100) -->
    <input type="number" min="1" max="100">

    <!-- Magia Wyrażeń Regularnych (Regex). Np. kod pocztowy w Polsce (XX-XXX) -->
    <input type="text" pattern="[0-9]{2}-[0-9]{3}" title="Podaj kod w formacie 00-000">

    <button type="submit">Wyślij</button>
</form>

```

27.2. Stylowanie błędów formularza w CSS

CSS posiada specjalne pseudoklasy (stany), które reagują na to, czy input jest aktualnie poprawny, czy błędny. Dzięki temu możemy rysować czerwoną ramkę automatycznie!

```

/* Domyślny wygląd pola */
input {
    border: 2px solid #ccc;
    padding: 10px;
    border-radius: 4px;
    outline: none; /* Ukrywamy brzydki systemowy focus */
}

/* Kiedy użytkownik zaczyna pisać i robi to DOBRZE (np. podał dobry kod pocztowy) */
input:valid {
    border-color: green;
}

```



```

/* Kiedy użytkownik pisze ŻLE (np. podał zły kod pocztowy) */
/* Dodajemy :focus, żeby nie świeciło na czerwono od razu po wejściu na pustą stronę! */
input:invalid:focus {
  border-color: red;
  background-color: #fff0f0;
}

```

27.3. Formularze w React (Controlled Components)

W React najczęściej przejmujemy kontrolę nad formularzem. Zamiast pozwolić HTML-owi robić swoje, zapisujemy wszystko, co wpisuje użytkownik, do stanu (State).

```

import React, { useState } from 'react';

export function ReactForm() {
  // 1. Tworzymy stan dla naszego pola tekstowego
  const [email, setEmail] = useState('');

  // 2. Funkcja wywoływana przy wciśnięciu przycisku
  const handleSubmit = (e) => {
    e.preventDefault(); // Blokuje domyślne odświeżenie strony przez formularz!
    alert(`Wysłano email: ${email}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>Adres E-mail:</label>
      <input
        type="email"
        required
        // 3. Wartość inputu jest "zamknięta" i połączona ze stanem
        value={email}
        // 4. Za każdym wciśnięciem klawisza aktualizujemy stan
        onChange={(e) => setEmail(e.target.value)}
      />
      <button type="submit">Wyślij</button>
    </form>
  );
}

```

28. SVG i Element Canvas

Do rysowania skomplikowanej grafiki na stronach służą dwie technologie: **SVG** (grafika wektorowa) i **Canvas** (grafika pikselowa kontrolowana z poziomu JS).

28.1. Wektorowe obiekty SVG

SVG nie traci na jakości przy powiększaniu. Idealnie nadaje się do ikon i logotypów. Można go wklejać bezpośrednio w HTML i sterować nim z CSS!

```

<!-- Rysowanie prostego okręgu -->
<!-- viewBox to nasze wirtualne płótno od 0 0 do 100 100 -->

```

```
<svg viewBox="0 0 100 100" width="200px" class="my-svg">
  <!-- cx, cy to środek, r to promień -->
  <circle cx="50" cy="50" r="40" fill="transparent" stroke="blue" stroke-width="5" />
</svg>
```

```
/* Możemy zmienić kolor SVG za pomocą zwykłego CSS! */
.my-svg circle {
  transition: stroke 0.3s ease;
}
.my-svg:hover circle {
  stroke: red; /* Kółko zmieni kolor na czerwony po najechaniu! */
}
```

28.2. Canvas (Płótno w JavaScript)

Płótno `<canvas>` to po prostu pusty kwadrat. Rysowanie po nim wymaga napisania logiki w JavaScript. Świetnie nadaje się do gier lub generowania wykresów.

```
import React, { useRef, useEffect } from 'react';

export function DrawingCanvas() {
  // Referencja do znacznika canvas (by mieć do niego dostęp w JS)
  const canvasRef = useRef(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    const ctx = canvas.getContext('2d'); // Pobieramy wirtualny pędzel

    // Rysujemy kwadrat
    ctx.fillStyle = 'green';
    ctx.fillRect(10, 10, 150, 100); // (x, y, szerokość, wysokość)

    // Rysujemy tekst
    ctx.font = '20px Arial';
    ctx.fillStyle = 'black';
    ctx.fillText('Witaj w Canvas!', 10, 150);
  }, []);

  return <canvas ref={canvasRef} width="400" height="200" style={{border: '1px solid black'}} />;
}
```

29. Wydajność (Performance) i Dostępność (A11y)

Nawet najładniejsza strona jest bezużyteczna, jeśli łąduje się 10 sekund albo uniemożliwia nawigację osobom niedowidzącym lub korzystającym wyłącznie z klawiatury.

29.1. Wydajność: Ładowanie Fontów (Font Display)

Kiedy przeglądarka widzi niestandardową czcionkę (np. z Google Fonts), domyślnie **ukrywa tekst**, dopóki plik z czcionką się nie pobierze (FOIT - Flash of Invisible Text). To irytuje użytkowników.

Dlatego zawsze powinniśmy stosować dyrektywę `font-display: swap;`.

```
@font-face {
  font-family: 'MojaCzcionka';
  src: url('/fonts/moja-czcionka.woff2') format('woff2');

  /* Magia! Przeglądarka od razu wyświetli systemowy Arial,
  aby użytkownik mógł od razu czytać tekst. Kiedy nasz plik się pobierze,
  podmieni (swap) wygląd na właściwy! */
  font-display: swap;
}
```

29.2. Wydajność: Animacje sprzętowe (GPU)

Pamiętaj złotą zasadę wydajnego interfejsu: **animuj tylko** `transform` i `opacity`.

Jeśli spróbujesz animować marginesy, szerokość (`width`) czy wysokość okna za pomocą CSS, zmusisz komputer do ciągłego przeliczania układu całej strony na nowo w każdej klatce (zjawisko Reflow/Layout). Klatkowanie na komórkach gwarantowane. Animacje `transform` (np. `scale`, `translate`) odbywają się w pamięci karty graficznej, nie wpływając na resztę układu!

```
/* ZLE - Zmusza procesor (CPU) do pracy przy każdej klatce */
.bad-button:hover {
  margin-top: -10px;
}

/* DOBRZE - Wykorzystuje akcelerację karty graficznej (GPU) */
.good-button:hover {
  transform: translateY(-10px);
}
```

29.3. Dostępność (A11y) w pigułce

A11y to skrót od *Accessibility* (słowo ma 11 liter pomiędzy a i y). Tworzenie dostępnego kodu to dzisiaj standard (często wymuszony prawnie przez dyrektywy unijne, np. WCAG).

1. Semantyka to podstawa: Nie rób przycisków z tagu `<div>`. Czytniki ekranowe (dla osób niewidomych) nie wiedzą, że `<div>` da się kliknąć. Używaj `<button>`.

```
// BŁĄD! Czytnik pominie ten element. Użytkownik korzystający z klawiatury (Tab) tu nie wejdzie.
<div className="btn" onClick={submit}>Wyślij</div>

// POPRAWNIE
<button className="btn" onClick={submit}>Wyślij</button>
```

2. Focus (zjawisko Outline): Często usuwamy brzydką, niebieską ramkę z inputów (`outline: none;`). To potężny błąd. Osoby poruszające się po stronie za pomocą klawisza `Tab` muszą wiedzieć, gdzie są! Jeśli bardzo chcesz ukryć ramkę po kliknięciu myszką, użyj pseudo-klasy `:focus-visible`.

```
/* Ukrywamy obrys dla kliknięcia myszką... */
button:focus:not(:focus-visible) {
  outline: none;
}
```

```
/* ...ale ZOSTAWIAMY wyraźny obrys, gdy użytkownik wszedł tu klawiszem TAB! */
button:focus-visible {
  outline: 3px dashed #007bff;
  outline-offset: 4px;
}
```

3. Atrybuty ARIA: Jeśli musisz stworzyć w React skomplikowany, własny element (np. suwak czy własny "Accordion"), musisz go odpowiednio "opisać" dla czytnika ekranowego za pomocą atrybutów `aria-*`.

```
export function CustomAlert({ message }) {
  return (
    // Rola "alert" informuje czytnik ekranowy, żeby NATYCHMIAST
    // przerwał czytanie czegokolwiek innego i przeczytał na głos ten błąd!
    <div role="alert" aria-live="assertive" className="error-box">
      {message}
    </div>
  );
}
```

Dokumentacja stworzona z myślą o nowoczesnych standardach React, HTML5 oraz potężne modułów CSS.

30. Budowa Własnego UI Kit w React (Tailwind CSS)

W tej sekcji wykorzystamy zdobytą wiedzę o HTML i CSS, aby stworzyć od zera własną bibliotekę komponentów (UI Kit) dla projektów React, korzystając z Tailwind CSS. Zamiast obszernej teorii, skupiamy się tutaj na czystym, profesjonalnym kodzie, gotowym do wdrożenia na produkcję.

30.1. Uniwersalny Przycisk (Button) z Wariantami

Profesjonalny przycisk musi obsługiwać różne stany (kolory) oraz zachowania (np. stan ładowania - `isLoading`).

```
// Button.jsx
import React from 'react';

export function Button({
  children,
  variant = 'primary',
  size = 'md',
  isLoading = false,
  disabled = false,
  onClick
}) {
  // Definiowanie klas bazowych (wspólnych dla wszystkich wariantów)
  const baseClasses = "inline-flex items-center justify-center font-medium rounded transition-colors duration-200 focus:outline-
none focus:ring-2 focus:ring-offset-2";

  // Obiekt słownikowy dla kolorów
  const variants = {
    primary: "bg-blue-600 hover:bg-blue-700 text-white focus:ring-blue-500",
    secondary: "bg-gray-200 hover:bg-gray-300 text-gray-800 focus:ring-gray-500",
    danger: "bg-red-600 hover:bg-red-700 text-white focus:ring-red-500",
  };
}
```

```

        ghost: "bg-transparent hover:bg-gray-100 text-gray-700"
    };

    // Obiekt słownikowy dla rozmiarów (Padding i Font Size)
    const sizes = {
        sm: "px-3 py-1.5 text-sm",
        md: "px-4 py-2 text-base",
        lg: "px-6 py-3 text-lg"
    };

    // Składanie ostatecznego łańcucha klas (String)
    const classes = `
        ${baseClasses}
        ${variants[variant]}
        ${sizes[size]}
        ${disabled || isLoading ? 'opacity-50 cursor-not-allowed' : ''}
    `;

    return (
        <button
            className={classes}
            disabled={disabled || isLoading}
            onClick={onClick}
        >

            {/* Prosty spinner ładowania wewnątrz przycisku wykonany w czystym SVG i obracający się przez animate-spin z Tailwinda */}

            {isLoading && (
                <svg className="animate-spin -ml-1 mr-2 h-4 w-4 text-current" xmlns="http://www.w3.org/2000/svg" fill="none"
viewBox="0 0 24 24">
                    <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor" strokeWidth="4"></circle>
                    <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962
7.962 0 014 12H0c0 3.042 1.135 5.824 3 7.938l3-2.647z"></path>
                </svg>
            )}
            {children}
        </button>
    );
}

```

Użycie komponentu Button:

```

<Button variant="primary" size="lg" onClick={() => console.log('Click!')}>
    Zapisz ustawienia
</Button>

<Button variant="danger" isLoading={true}>
    Usuwanie...
</Button>

```

30.2. W pełni Dostępny Modal (Okno Dialogowe)

Budowa modala to trudne zadanie, ponieważ musimy przykryć całą resztę aplikacji (Z-Index), zablokować przewijanie tła strony (body overflow) i obsłużyć czytniki ekranowe (role="dialog").

```

// Modal.jsx
import React, { useEffect } from 'react';

export function Modal({ isOpen, onClose, title, children }) {
  // Kiedy modal jest otwarty, blokujemy możliwość scrollowania głównej strony
  useEffect(() => {
    if (isOpen) {
      document.body.style.overflow = 'hidden';
    } else {
      document.body.style.overflow = 'unset';
    }
    // Cleanup function (wywoływane przy odmontowaniu)
    return () => {
      document.body.style.overflow = 'unset';
    };
  }, [isOpen]);

  // Jeśli isOpen jest false, nie renderujemy w ogóle kodu HTML
  if (!isOpen) return null;

  return (
    // Overlay (przyciemnione tło obejmujące cały ekran)
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black bg-opacity-50 backdrop-blur-sm transition-opacity">

      {/* Pudełko samego Modala. Używamy role="dialog" dla Dostępności (A11y) */}
      <div
        role="dialog"
        aria-modal="true"
        aria-labelledby="modal-title"
        className="bg-white rounded-xl shadow-2xl w-full max-w-md overflow-hidden transform transition-all"
      >
        {/* Header Modala (Tytuł i krzyżyk zamknięcia) */}
        <div className="flex justify-between items-center p-5 border-b border-gray-100">
          <h3 id="modal-title" className="text-lg font-semibold text-gray-900">
            {title}
          </h3>
          <button
            onClick={onClose}
            className="text-gray-400 hover:text-gray-600 focus:outline-none focus:ring-2 focus:ring-blue-500 rounded-full p-1"
            aria-label="Zamknij okno"
          >
            {/* Ikona X (Close) jako SVG */}
            <svg className="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M6 18L18 6M6 6L18 18" />
            </svg>
          </button>
        </div>

        {/* Główne ciało Modala, gdzie przekażemy formularze lub tekst */}
        <div className="p-5 text-gray-700">
          {children}
        </div>
      </div>
    </div>
  );
}

```

```

        </div>
      </div>
    </div>
  );
}

```

Użycie komponentu Modal:

```

// App.jsx
import React, { useState } from 'react';
import { Modal } from './Modal';
import { Button } from './Button';

export default function App() {
  const [isModalOpen, setIsModalOpen] = useState(false);

  return (
    <div className="p-10">
      <Button onClick={() => setIsModalOpen(true)}>
        Otwórz ustawienia
      </Button>

      <Modal
        isOpen={isModalOpen}
        onClose={() => setIsModalOpen(false)}
        title="Ustawienia Konta"
      >
        <p className="mb-4">Czy na pewno chcesz zmienić plan subskrypcji na wyższy?</p>
        <div className="flex justify-end gap-3">
          <Button variant="ghost" onClick={() => setIsModalOpen(false)}>
            Anuluj
          </Button>
          <Button variant="primary" onClick={() => alert('Zmieniono!')}>
            Potwierdź
          </Button>
        </div>
      </Modal>
    </div>
  );
}

```

30.3. Pole tekstowe z etykietą i błędem (InputField)

Kolejny kluczowy element UI Kit to pole formularza. Zamiast za każdym razem powtarzać strukturę `Label` → `input` → `error`, zamykamy ją w jednym sprytnym komponencie.

```

// InputField.jsx
import React from 'react';

export function InputField({
  label,
  id,

```

```

    type = 'text',
    error,
    value,
    onChange,
    placeholder,
    required = false
  }) {
    return (
      <div className="flex flex-col mb-4">
        {/* Etykieta formularza powiązana z inputem poprzez atrybut htmlFor */}
        <label htmlFor={id} className="mb-1 text-sm font-medium text-gray-700">
          {label} {required && <span className="text-red-500">*</span>}
        </label>

        <input
          id={id}
          type={type}
          value={value}
          onChange={onChange}
          placeholder={placeholder}
          required={required}
          // Atrybuty A11y (Dostępność)
          aria-invalid={error ? "true" : "false"}
          aria-describedby={error ? `${id}-error` : null}
          className={`
            px-4 py-2 border rounded-lg focus:outline-none focus:ring-2 transition-colors
            ${error
              ? 'border-red-300 focus:ring-red-500 bg-red-50'
              : 'border-gray-300 focus:ring-blue-500 focus:border-blue-500'}
            `}
        />

        {/* Warunkowe renderowanie czerwonego komunikatu o błędzie poniżej inputu */}
        {error && (
          <p id={` ${id}-error`} className="mt-1 text-sm text-red-600">
            {error}
          </p>
        )}
      </div>
    );
  }
}

```

Użycie InputField:

```

// Gdzieś w formularzu
<InputField
  id="email"
  label="Adres E-mail"
  type="email"
  value={userEmail}
  onChange={(e) => setUserEmail(e.target.value)}
  error={emailError} // Np. "Podano niepoprawny format adresu"

```



```
required={true}  
</>
```

31. Zaawansowane Komponenty (Dropdown, Accordion, Zakładki)

Kiedy budujemy zaawansowane interfejsy zagnieżdżone, najważniejszym aspektem jest łączenie zarządzania stanem w React (zmienne typu boolean) z płynnymi przejściami CSS.

31.1. W pełni funkcjonalny Dropdown (Rozwijane Menu)

Dropdown to komponent, w którym musimy śledzić kliknięcia użytkownika nie tylko wewnątrz samego menu, ale także *poza nim* (aby zamknąć menu, gdy użytkownik kliknie gdziekolwiek indziej na ekranie). Wykorzystujemy do tego tzw. nasłuchiwanie na obiekcie globalnym (window event listener) oraz referencję `useRef`.

```
import React, { useState, useRef, useEffect } from 'react';  
  
export function DropdownMenu() {  
  // Stan określający, czy menu jest otwarte, czy zamknięte  
  const [isOpen, setIsOpen] = useState(false);  
  
  // Referencja na cały komponent, by wiedzieć, gdzie fizycznie kliknął użytkownik  
  const dropdownRef = useRef(null);  
  
  useEffect(() => {  
    // Funkcja sprawdzająca czy kliknięcie nastąpiło POZA komponentem  
    const handleClickOutside = (event) => {  
      if (dropdownRef.current && !dropdownRef.current.contains(event.target)) {  
        setIsOpen(false);  
      }  
    };  
  
    // Nasłuchujemy na każde kliknięcie myszą w całej aplikacji  
    document.addEventListener('mousedown', handleClickOutside);  
  
    return () => {  
      // Sprzątanie po odmontowaniu komponentu, bardzo ważne dla wydajności!  
      document.removeEventListener('mousedown', handleClickOutside);  
    };  
  }, []);  
  
  return (  
    <div className="relative inline-block text-left" ref={dropdownRef}>  
      /* Przycisk otwierający/zamykający */  
      <button  
        onClick={() => setIsOpen(!isOpen)}  
        className="inline-flex justify-center w-full rounded-md border border-gray-300 shadow-sm px-4 py-2 bg-white text-sm font-medium text-gray-700 hover:bg-gray-50 focus:outline-none"  
      >  
        Opcje  
        /* Ikonka strzałki, która się obraca gdy menu jest otwarte */  
        <svg  
          className={`mr-1 ml-2 h-5 w-5 transform transition-transform duration-200 ${isOpen ? 'rotate-180' : ''}`}  
          xmlns="http://www.w3.org/2000/svg" viewBox="0 0 20 20" fill="currentColor"
```

```

        >
        <path fillRule="evenodd" d="M5.293 7.293a1 1 0 011.414 0L10 10.586L3.293-3.293a1 1 0 111.414 1.414L-4 4a1 1 0 01-1.414 0L-4-4a1 1 0 010-1.414z" clipRule="evenodd" />
      </svg>
    </button>

    {/* Warunkowe renderowanie: Jeśli isOpen to prawda, pokaż menu */}
    {isOpen && (
      <div className="origin-top-right absolute right-0 mt-2 w-56 rounded-md shadow-lg bg-white ring-1 ring-black ring-opacity-5 focus:outline-none z-10 animate-fade-in-down">
        <div className="py-1" role="menu">
          <a href="#" className="text-gray-700 block px-4 py-2 text-sm hover:bg-gray-100" role="menuitem">Edytuj
Profil</a>
          <a href="#" className="text-gray-700 block px-4 py-2 text-sm hover:bg-gray-100" role="menuitem">Ustawienia
Konta</a>
          <a href="#" className="text-red-600 block px-4 py-2 text-sm hover:bg-red-50 mt-2 border-t"
role="menuitem">Wyloguj się</a>
        </div>
      </div>
    )}
  </div>
);
}

```

```

/* Plik custom-animations.css, dołączony np. w global.css lub konfigu Tailwinda */
@keyframes fadeInDown {
  0% {
    opacity: 0;
    transform: translateY(-10px) scale(0.95);
  }
  100% {
    opacity: 1;
    transform: translateY(0) scale(1);
  }
}

.animate-fade-in-down {
  animation: fadeInDown 0.2s ease-out forwards;
}

```

31.2. Accordion (Harmonijka Q&A) z Płynnym CSS

Stworzenie rozwijanej zakładki (np. w sekcji FAQ - Najczęściej zadawane pytania), która płynnie rozsuwa się w dół, było bardzo trudne w CSS ze względu na to, że wysokość zależy od zawartości (`height: auto`). Klasyczne przejścia CSS nie potrafią animować wartości `auto`.

Zastosujemy sztuczkę z `max-height`.

```

import React, { useState } from 'react';

// Pojedynczy element harmonijki
function AccordionItem({ title, content }) {

```

```

const [isOpen, setIsOpen] = useState(false);

return (
  <div className="border-b border-gray-200">
    <button
      onClick={() => setIsOpen(!isOpen)}
      className="flex justify-between w-full py-4 text-left focus:outline-none"
    >
      <span className="font-medium text-gray-900">{title}</span>
      <span className="text-blue-500 text-xl font-bold transition-transform duration-300 transform">
        {isOpen ? '-' : '+'}
      </span>
    </button>

    {/*
      Tu dzieje się magia! Zamiast conditional renderingu ( {isOpen && <div> } ),
      używamy klas CSS, aby kontrolować max-height.
      Gdy jest zamknięte, max-height = 0. Gdy otwarte = 1000px.
    */}
    <div
      className={`overflow-hidden transition-all duration-300 ease-in-out ${
        isOpen ? 'max-h-96 opacity-100 pb-4' : 'max-h-0 opacity-0'
      }`}
    >
      <p className="text-gray-600">
        {content}
      </p>
    </div>
  </div>
);
}

// Główny komponent grupujący
export function FaqAccordion() {
  const faqs = [
    { id: 1, q: "Czym jest React?", a: "React to biblioteka do budowania UI." },
    { id: 2, q: "Czy Tailwind jest darmowy?", a: "Tak, to projekt open-source." },
    { id: 3, q: "Jak uczyć się programowania?", a: "Pisać dużo własnego kodu!" },
  ];

  return (
    <div className="max-w-xl mx-auto mt-10 bg-white p-6 rounded-lg shadow-md">
      <h2 className="text-2xl font-bold text-gray-800 mb-6">FAQ</h2>
      {faqs.map(faq => (
        <AccordionItem key={faq.id} title={faq.q} content={faq.a} />
      ))}
    </div>
  );
}

```

31.3. Tabs (Zarządzanie Stanem Indexu)

Zakładki to klasyczny przypadek renderowania warunkowego opartego o jeden wspólny stan (najczęściej jest to Index aktywnej zakładki).

```

import React, { useState } from 'react';

export function TabController() {
  // Przechowujemy index aktualnie otwartej karty. Domyślnie 0 (pierwsza).
  const [activeTabIndex, setActiveTabIndex] = useState(0);

  // Nasze dane
  const tabsData = [
    { title: "Profil", content: "To są dane profilowe użytkownika." },
    { title: "Historia Zamówień", content: "Kupiłeś u nas 3 przedmioty w zeszłym roku." },
    { title: "Ustawienia Prywatności", content: "Możesz zmienić swoje hasło poniżej." }
  ];

  return (
    <div className="w-full max-w-2xl mx-auto mt-10 border rounded-lg overflow-hidden bg-white shadow-sm">

      {/* Pasek Nawigacyjny Zakładek */}
      <div className="flex border-b bg-gray-50">
        {tabsData.map((tab, index) => (
          <button
            key={index}
            onClick={() => setActiveTabIndex(index)}
            className={`flex-1 py-3 px-4 text-center font-medium text-sm transition-colors duration-200 focus:outline-
none ${
              activeTabIndex === index
                ? 'bg-white border-b-2 border-blue-500 text-blue-600'
                : 'text-gray-500 hover:text-gray-700 hover:bg-gray-100'
            }`}
          >
          {tab.title}
        </button>
        ))}
      </div>

      {/* Treść (Renderuje tylko content przypisany do aktualnego Indexu) */}
      <div className="p-6 text-gray-700 bg-white min-h-[200px]">
        {/* Dodajemy animację wchodzenia po przełączeniu */}
        <div key={activeTabIndex} className="animate-fade-in-down">
          {tabsData[activeTabIndex].content}
        </div>
      </div>
    </div>
  );
}

```

31.4. Podsumowanie Wzorców

Powyższe komponenty reprezentują wyższy poziom inżynierii Front-End:

1. **Button** korzysta ze "słowników" (obiektów JS) zamiast ogromnych kaskad `if-else` do wyboru stylów wariantu.
2. **Modal** prawidłowo odcina aplikację pod spodem (body overflow) i dba o `role="dialog"` dla dostępności (a11y).

3. **Dropdown** pokazuje, jak wykorzystać wbudowane w przeglądarkę zdarzenia globalne (`window.addEventListener`) do wykrywania kliknięć zewnętrznych.
4. **Accordion** prezentuje, jak w elegancki sposób obejść ograniczenia silników renderujących CSS, korzystając z przejść na `max-height`.
5. **Tabs** obrazuje, jak tablice w JavaScript pozwalają na wysoce dynamiczne iterowanie (`map()`) i powiązanie z centralnym punktem prawdy w `useState`.

Budując nowoczesne aplikacje, zawsze miej na uwadze semantykę (właściwe tagi HTML), czystą kaskadę lub utility-classes (CSS) oraz jednokierunkowy przepływ danych (React).

32. Zaawansowana Kontrola Rozmieszczenia (Stacking Context)

Jednym z najczęstszych problemów w CSS jest `z-index`. Zdarza się, że element z `z-index: 9999` wciąż chowa się pod elementem z `z-index: 1`. Wynika to z tzw. **Kontekstu Stosu (Stacking Context)**.

Z-index nie jest wartością absolutną dla całej strony. Działa on tylko w obrębie swojego lokalnego kontekstu.

32.1. Jak tworzy się Stacking Context?

Kontekst stosu jest tworzony, gdy element ma m.in.:

1. `position: absolute/relative/fixed/sticky` ORAZ `z-index` różny od `auto`.
2. `opacity` mniejsze niż 1.
3. Właściwość `transform`, `filter`, `backdrop-filter` lub `clip-path` inną niż domyślna.
4. Kontenery CSS Grid i Flexbox, których dzieci mają `z-index` różny od `auto`.

```
.parent-a {
  position: relative;
  z-index: 1; /* Tworzy nowy kontekst stosu! */
}

.child-a {
  position: absolute;
  z-index: 9999; /* Wydaje się ogromne, ale z perspektywy strony to wciąż tylko "Dziecko Parenta A (który ma 1)" */
}

.parent-b {
  position: relative;
  z-index: 2; /* Parent B ma z-index 2. Jest WYŻEJ na stronie niż Parent A (który ma 1) */
}

/* WNIÓSEK: Parent B ZAWSZE przykryje Child A, niezależnie od tego, że Child A ma 9999! */
```

32.2. CSS Sticky (Lepkie Pozycjonowanie)

Właściwość `position: sticky` to hybryda między pozycjonowaniem relatywnym a stałym (fixed). Element przewija się ze stroną, aż osiągnie podany próg na ekranie, a potem się "przykleja".

```
export function StickySidebarLayout() {
  return (
    <div className="flex max-w-5xl mx-auto gap-8 mt-10">
      {/* Lewa kolumna z głównym długim tekstem */}
      <main className="w-2/3 pb-96">
        <h1 className="text-3xl font-bold mb-4">Główny Artykuł</h1>
        <p>Bardzo, bardzo długi tekst...</p>
      </main>
    </div>
  )
}
```

```

        <div className="h-[2000px] bg-gray-100 rounded-lg mt-4">Puste pole wymuszające przewijanie okna</div>
    </main>

    /* Prawa kolumna (Sidebar) */
    <aside className="w-1/3">
        /* Ten element będzie przewijał się ze stroną, ale ZATRZYMA SIĘ 20px od górnej krawędzi okna! */
        <div className="sticky top-5 bg-white p-6 shadow-md rounded-lg border border-gray-200">
            <h2 className="text-xl font-bold mb-2">Spis Treści</h2>
            <ul className="space-y-2 text-blue-600">
                <li>Wprowadzenie</li>
                <li>Rozwinięcie</li>
                <li>Zakończenie</li>
            </ul>
            <button className="mt-4 w-full bg-blue-600 text-white py-2 rounded hover:bg-blue-700">
                Kup Książkę
            </button>
        </div>
    </aside>
</div>

);
}

```

Uwaga: Aby `position: sticky` zadziałało, rodzic nie może mieć ukrytego przepełnienia (nie może posiadać `overflow: hidden`). To najczęstszy powód niedziałającego sticky!

33. Scroll-Driven Animations (Animacje Napędzane Scrollowaniem)

To absolutna nowość w przeglądarkach (częściowo wymaga jeszcze tzw. polyfilli lub najnowszych wersji Chrome). Pozwala powiązać CSS-ową animację `@keyframes` nie z czasem (sekundami), ale bezpośrednio z paskiem przewijania na ekranie.

33.1. Pasek postępu czytania (Reading Progress Bar)

Zamiast pisać 50 linii kodu w JavaScript do nasłuchiwania `window.addEventListener('scroll')`, możemy to zrobić w samym CSS!

```

/* Klasyczna animacja rośnięcia od 0% do 100% szerokości */
@keyframes grow-progress {
    from { transform: scaleX(0); }
    to   { transform: scaleX(1); }
}

.reading-progress-bar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 8px;
    background-color: #ef4444; /* Czerwony kolor */

    /* Zaczepiamy punkt skalowania po lewej stronie, żeby rósł z lewej do prawej */
    transform-origin: 0 50%;

    /* Zamiast "2s linear", używamy "scroll()" ! */
    animation: grow-progress linear;
}

```

```
    animation-timeline: scroll(root block);  
  }  
}
```

```
// Gdzieś na samym szczycie struktury aplikacji  
export function Layout({ children }) {  
  return (  
    <div>  
      {/* Pasek śledzi scrollowanie okna i rośnie z automatu */}  
      <div className="reading-progress-bar" />  
  
      <main>  
        {children}  
      </main>  
    </div>  
  );  
}
```